

1. INTRODUCTION

1.1 OVERVIEW OF THE SYSTEM

Agriculture in modern times may or may not be much easier depending upon the crop health. Farming of wheat, rice, tomatoes, potatoes, and specialized crops are more difficult forms of agriculture. Farming can also be more difficult depending on the environmental conditions, such as temperature, humidity, or even by pest attacks.

A plant disease detection system is a tool designed to help farmers or agricultural organizations monitor, categorize, and control their crop-related diseases. Coupled with classification techniques, it provides a clear and comprehensible way to understand infection patterns and make informed farming decisions.

This idea proposes a function to calculate the health index of your crop, taking into account factors like leaf spot symptoms. You simply upload the photo of your leaf, and it'll provide you with an estimate of which disease is affecting the plant. Also offer personalized recommendations for remedies and preventive measures along your location. Whether you're dealing with a bacterial infection or in need of a biological control method, our app has got your back. It'll suggest the best options based on your preferences and crop type.

The project includes modules by authentication, admin, detection, dashboard. Admin manages the user accounts, including registration, authentication, and user profile. The detection module provides a real-time abstract of crops & diseases and their severity. User are the farmers or the ones that are cultivating.

1.2 PROBLEM DEFINITION AND OBJECTIVE OF THE PROJECT

Plant disease detection systems can help farmers and agricultural organizations manage crop health in a variety of ways, including:

- o **Monitoring:** Farmers can monitor their crops and avoid yield losses. Organizations can track disease outbreaks and make informed management decisions.
- o **Treatment:** Agricultural workers can identify diseases to submit diagnosis reports and claim treatment support.
- o **Prevention:** Some crop diseases are preventable, so detailed records are important for warning purposes.
- o **Agricultural planning:** Understanding crop diseases helps farmers plan future cultivations more effectively.

Plant disease detection systems help individuals and organizations keep track of their crop health. For farms, they help record crop infections. This helps with processing treatments, managing yields, and taking advantage of support. Crop health reports also help farmers stay within budget by analyzing infection patterns.

Some examples of crop parameters that can be monitored include:

- Visual symptoms: Leaf spots, leaf blight, rust, powdery mildew, and mosaic virus
- Environmental metrics: Temperature, humidity, precipitation, and soil moisture
- Treatments: Pesticides, fungicides, organic remedies, and soil conditioners
- Interventions: Irrigation adjustments, fertilization, and crop rotation schedules

The project includes modules by authentication, admin, detection, dashboard. Admin manages the user accounts, including registration, authentication, and user profile. The detection module provides a real-time abstract of crops & diseases and their severity. User are the farmers or the ones that are cultivating.

2. SYSTEM ANALYSIS

2.1 INTRODUCTION

System analysis of study is an important phase of any system development process. The system is studied to the minutest details and analysed. The system analyst plays the role of an interrogator and dwells deep into the working of the present system. The system is viewed as a whole and the inputs to the system are identified. The outputs from the organization are traced through the various processing. A detailed study of this process must be made by various techniques like interviews, questionnaires etc. The data collected by these sources must be scrutinized to arrive at a conclusion. The conclusion is an understanding of the system function.

2.2 IDENTIFICATION OF NEED

Software engineering is the analysis, design, construction, verification and management of technical or social entities. The work associated with software engineering can be categorized into three generic phases. The system requirement analysis is done by the user in order to meet the required project. The goal is to introduce an application that facilitates and provides faster and more accurate methods for plant disease diagnosis that can replace traditional manual inspection techniques. This project basically includes the requirements for providing automated plant disease detection, multilingual diagnostic support, and comprehensive agricultural monitoring dashboards.

2.3 EXISTING SYSTEM

There are numerous systems that support plant disease identification but not all of them perform edge-first local inference, dynamic multilingual translation, and text-to-speech audio guidance. The features are limited and often require constant internet connectivity. Some provide basic image classification but lack comprehensive treatment recommendations, weather-based risk assessment, and membership-based access controls.

2.3.1 LIMITATION OF EXISTING SYSTEM

The existing system has various limitations compared to the proposed system. Some of the limitations are listed below:

1. User couldn't often perform diagnosis without active internet connection.
2. If translation APIs hit rate limits, non-English speaking farmers could not understand diagnostic recommendations.

2.4 PROPOSED SYSTEM

This project proposes a function to identify crop diseases, provide multilingual translation, text-to-speech guidance and local weather monitoring. To maintain the crop's safety from the start of cultivation till the end of harvest this idea can be implemented and keep them safe and sound. Also the diagnostic report can be shared with nearest agricultural departments to make it easier for anyone available to come for assistance.

2.4.1 ADVANTAGES OF PROPOSING SYSTEM

1. Fastest access: If they upload an image, they can access diagnosis fastly, they can access the local model and identify plant diseases.
2. Multilingual guidance: This app provides translation to nearest regional languages like Malayalam, Hindi etc., they can understand treatments fastly.

2.5 FEASIBILITY STUDY

All projects are feasible when given unlimited resources and infinite time. It is both necessary and prudent to evaluate the feasibility of a project at the earliest possible time. An estimate is made of whether the identified user needs may be satisfied using current software and hardware technologies. The study will decide if the proposed system will be cost effective from the business point of view and if it can be developed within the given existing budgetary constraints.

2.5.1 TECHNICAL FEASIBILITY

Technical feasibility study deals with the hardware as well as software requirements. The scope was whether the work for the project is done with the current equipment and the existing software technology. The system requires Python 3.10+, Django 6.0, TensorFlow/Keras, and SQLite3 database. Since these are all freely available open-source technologies, this system is technically feasible.

2.5.2 ECONOMIC FEASIBILITY

The purpose of economic feasibility assessment is to determine the positive benefit to the organization that the proposed system will provide. Our system is economically feasible since it eliminates cloud API subscription costs by deploying local inference and keyless translation brokers. It is highly cost effective with zero operational overhead.

2.5.3 OPERATIONAL FEASIBILITY

The purpose of operational feasibility study is to determine whether the new system will be used if it is developed and implemented. Our system will operate after it is developed and be operative once installed. The user interface features single-click actions and clear visual previews for uploads. Spoken audio translation guides allow non-literate farmers to use the system easily. This system is operationally feasible.

2.5.4 SCHEDULE FEASIBILITY

Schedule feasibility is a measure of how reasonable is the project time table. Each phase of the system development should be done within a specific time frame to avoid delays in the implementation of testing of the final product.

2.6 SYSTEM SPECIFICATION

After the analyst has collected all required information regarding the software to be developed, and has removed all incompleteness, inconsistencies, and anomalies from the specification, the requirements are systematically organized in the form of an SRS document. The software developers refer to the SRS document to make sure that they developed exactly what the customer requires.

2.6.1 SOFTWARE SPECIFICATION

A software requirements specification (SRS) is a description of a software system to be developed. Software requirements specification establishes the basis for an agreement between customers and contractors or suppliers on how the software product should function. Software requirements specification is a rigorous assessment of requirements before the more specific system design stages, and its goal is to reduce later redesign.

- **HTML:** It is the standard mark-up language for documents designed to be displayed in a web browser. It can be assisted by CSS and JavaScript. Web browsers receive HTML documents from a web server or from local storage and render the documents into multimedia web pages. HTML describes the structure of a web page semantically. HTML elements are the building blocks of HTML pages. With HTML constructs, images and other objects may be embedded. Browsers do not display the HTML tags but use them to interpret the content of the page.

Front End (Continued)

- **Cascading Style Sheets (CSS) / Tailwind CSS:** CSS is a style sheet language used to describe the presentation of a document written in HTML or XML. CSS describes how elements should be rendered on screen, on paper, in speech, or on other media. The project uses Tailwind CSS utility-first framework for rapid, responsive UI development with modern design patterns.

Back End

- **Python / Django:** Python is a high-level, general-purpose programming language known for its readability and versatility. Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. It follows the model-template-views (MTV) architectural pattern.

Django provides a structural foundation for building secure, scalable web applications. The framework includes built-in solutions for common tasks such as database routing, cookie-based session handling, user account authorization, and template rendering. In this project, Django serves as the core orchestration layer, processing user actions, coordinating ML inference tasks, interfacing with SQLite3 databases, routing data between client views, and managing background network communications with translation and weather services.

Back End (Continued)

• Python / Django Features:

1. Server-side processing: Django handles all backend logic including OTP authentication, image processing, and database management on the server.
2. ORM System: Django's Object-Relational Mapping allows developers to interact with databases using Python objects instead of raw SQL queries.
3. Built-in Security: Django provides built-in protection against SQL injection, cross-site scripting, CSRF attacks, and clickjacking.
4. Extensible Architecture: Django supports modular app development with reusable components and a rich ecosystem of third-party packages.
5. Cross-platform Compatibility: Django applications can run on Windows, macOS, and Linux with Python 3.x support.

• Python / Django Advantages:

1. Rapid Development: Django's "batteries included" philosophy provides built-in admin, authentication, and ORM systems for fast prototyping.
2. Scalable: Django powers high-traffic websites and can scale from small projects to enterprise-level applications.
3. Large Community: Django has a massive community of developers with extensive documentation, tutorials, and third-party packages.
4. Cost-effective: Django is an open-source framework with no licensing costs.

• Python / Django Disadvantages:

1. Monolithic structure: Django's tightly coupled components can make it difficult to swap individual parts.
2. Learning curve: The framework's ORM and template system require dedicated learning for beginners.
3. Overhead: For very simple applications, Django's full-featured nature may introduce unnecessary complexity.

In conclusion, Python with Django is a powerful and versatile combination that is well-suited for building intelligent web applications. Its built-in security, ORM system, and rapid development capabilities make it an ideal choice for building this plant disease detection system.

Database and Frameworks

- **SQLite3:** SQLite is a lightweight, serverless, self-contained relational database management system (RDBMS). Relational databases are key components of Django applications. In this system, SQLite3 is used to store user membership logs, crop database mappings, and diagnosis parameters.
- **TensorFlow / Keras:** TensorFlow is an open-source machine learning framework developed by Google. Keras is a high-level API running on top of TensorFlow that provides a simple and intuitive interface for building and training deep learning models. The project uses TensorFlow/Keras to deploy a pre-trained MobileNetV2 model for plant disease classification with transfer learning from ImageNet weights.
- **Operating System:** Windows 10/11 or Ubuntu Linux provides the operating environment. The system supports cross-platform deployment with Python's platform-independent runtime. The operating system handles file I/O for image uploads, database operations, and model inference.

2.6.2 TOOLS SPECIFICATION

Tools: Google Chrome, Visual Studio Code, Python CLI

Platform: Windows / Linux

2.6.3 HARDWARE SPECIFICATION

Processor: Intel Core i3 or above (dual-core minimum)

RAM: 4GB minimum (8GB recommended for ML inference)

Hard Disc: 2GB minimum for model files and database

Display Type: PC Display / Mobile Display

Input Device: Mouse, Keyboard, Camera (for leaf image capture)

Output Device: Monitor, Mobile Display, Audio Speaker (for TTS playback)

3. SYSTEM DESIGN

3.1 INTRODUCTION

System design is an iterative process through which requirements are transmitted to a "blueprint" for constructing the software. Initially, the blueprint depicts a holistic view of software that is designed and represented at a high-level abstraction. A level that can be directly traced to specific data, functional and behavioural requirements. As design iterations occur, subsequent refinement leads to design representations at much lower levels of abstraction. System design is a creative art of inventing and developing inputs, databases, offline files, methods and procedures for processing data to get meaningful output that satisfies the organization's objectives. Through the design phase, consideration to the human factor, that is inputs to the users, will have on the system. Some of the main factors that have to be noted during the design of the system are: Practicability, Efficiency, Cost, and Security.

3.2 INPUT DESIGN

Input design is the process of converting user-oriented input to a computer-based format. Inaccurate input data are the most common cause of errors in data processing. Errors entered by data entry operators can be controlled by input design. The goal of designing input data is to make data entry as easy, logical and free from errors. When we approach input data design, we design the data source documents that capture the data and then select the media used to enter them into the computer. User-friendly screen formats can reduce the burden on end users. In this system, the primary inputs include leaf images uploaded by users, OTP codes for authentication, user profile information, and administrative resource management forms. The input validation ensures only valid image formats (JPG, JPEG, PNG) under 5MB are accepted for disease diagnosis.

3.3 OUTPUT DESIGN

Computer output is the most important and direct source of information to the user. Efficient and intelligent output design improves the system's relationship and helps user decision making. In this system, the outputs include disease prediction results with confidence scores, structured treatment guidance plans, multilingual translated diagnostic reports, audio playback of treatment recommendations, dashboard analytics charts, weather-based risk alerts, and administrative reports. The output design ensures that diagnostic results are presented in a clear, actionable format with visual indicators for disease severity and treatment urgency.

3.4 DATABASE DESIGN

Database design is a most important part of the system design phase. In a database environment, data available are used by several users instead of each program managing its own data. Authorized users share data across applications with the database software managing the data as an entity. Primary key is one of the candidate keys that are chosen to be the identifying key for the entire table. Normalization is a process of organizing the data in database to avoid data redundancy, insertion anomaly, update anomaly and deletion anomaly.

3.4 DATABASE DESIGN (Continued)

Normalization is a process of organizing the data in database to avoid data redundancy, insertion anomaly, update anomaly and deletion anomaly. Here are the most commonly used normal forms:

- **First normal form (1NF):** As per the rule of first normal form, an attribute (column) of a table cannot hold multiple values. It should hold only atomic values.
- **Second normal form (2NF):** A table is said to be in 2NF if both the following conditions hold: Table is in 1NF and no non-prime attribute is dependent on the proper subset of any candidate key.
- **Third normal form (3NF):** Dependency of non-prime attribute on any super key should be removed. A table is in 3NF if it is in 2NF and for each functional dependency $X \rightarrow Y$ at least one of the following conditions hold: X is a super key of table, or Y is a prime attribute of table.
- **Boyce Codd normal form (BCNF):** It is an advance version of 3NF. BCNF is stricter than 3NF. A table complies with BCNF if it is in 3NF and for every functional dependency $X \rightarrow Y$, X should be the super key of the table.

The database normalization ensures structural integrity and eliminates data redundancy. By ensuring that all tables comply with at least Third Normal Form (3NF), our Plant Disease Detection system minimizes storage overhead and protects against update anomalies. The relational schema is structured with clear foreign keys that link LeafDiagnosis records to Crops and Diseases, and MembershipProfiles to primary User records.

Below is a brief summary of how normalization principles were applied to the system schema design:

1. **1NF Application:** Every attribute contains only atomic values. For example, crop diseases and treatment recommendations are represented as structured text blocks rather than nested multi-value lists in a single field.
2. **2NF Application:** All non-prime attributes are fully dependent on the primary key, avoiding partial dependencies. Tables such as LeafDiagnosis are keyed by a unique diagnostic identifier.
3. **3NF Application:** Transitive functional dependencies are removed. The User model attributes do not depend on non-key columns of other entities.

3.4 DATABASE DESIGN (BCNF Application)

Boyce-Codd Normal Form (BCNF) requires that for every non-trivial functional dependency $X \rightarrow Y$, X must be a super key. In our database tables, such as Crop, Disease, User, and LeafDiagnosis, this rule is strictly adhered to, ensuring that there are no non-trivial dependencies on non-superkeys, thereby eliminating any potential for redundancy or inconsistency.

3.5 DATA FLOW DIAGRAM (DFD)

The Data Flow Diagram (DFD) also known as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of the input data to the system, various processing carried out on these data and the output data generated by the system. The main reason why the DFD technique is so popular is probably because of the fact that DFD provides a very simple formalism. It is simple to understand and use. A DFD model uses a very limited number of primitive symbols to represent the functions performed by a system and the data flow among these systems. Starting with a set of high-level functions that a system performs, the DFD model hierarchically represents various sub functions. The Data Flow Diagramming technique also follows a simple set of intuitive concepts and rules. Data flow diagrams are a way of expressing system requirements in a graphical manner. DFD represents one of the most ingenious tools used for structured analysis.

BASIC DFD SYMBOLS

External Entity Symbol: An external entity such as a user, admin etc. is represented by a rectangle. The external entities are essentially those physical entities external to the application system which interact with the system by inputting data to the system or by consuming the data produced by the system.

BASIC DFD SYMBOLS (Continued)

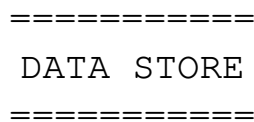
Process Symbol: The functions are represented using circles. Bubbles are annotated with the names of the corresponding functions. They convert data into information.



Data Flow Symbol: A directed arc or an arrow is used as a Data Flow Symbol. This represents the data flow occurring between two processes or between an external entity and a process in the direction of the Data Flow Arrow. Data flow symbols are annotated with corresponding data names.

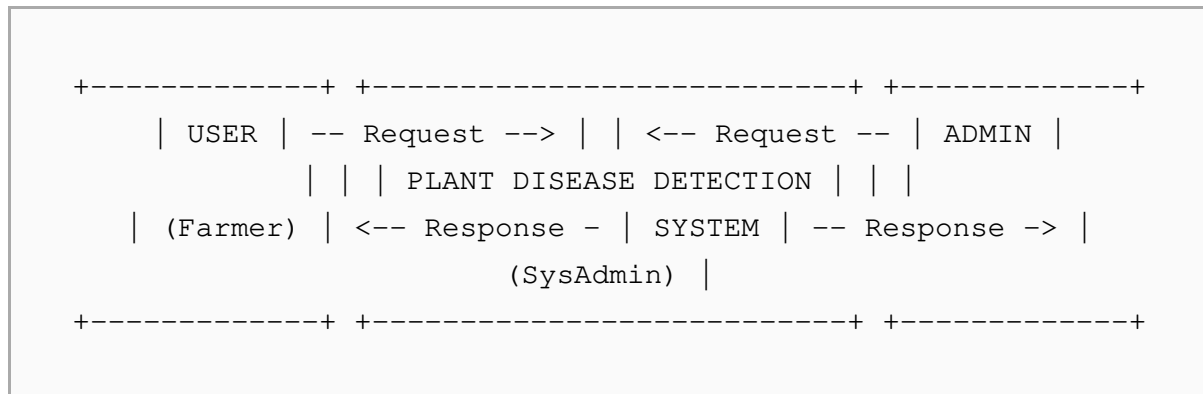


Data Store Symbol: A Data Store represents a logical file. It is represented using two parallel lines. A logical file can represent either a data structure or a physical file on disk. Each data store is connected to a process by means of a Data Flow Symbol.



DFD FOR PLANT DISEASE DETECTION

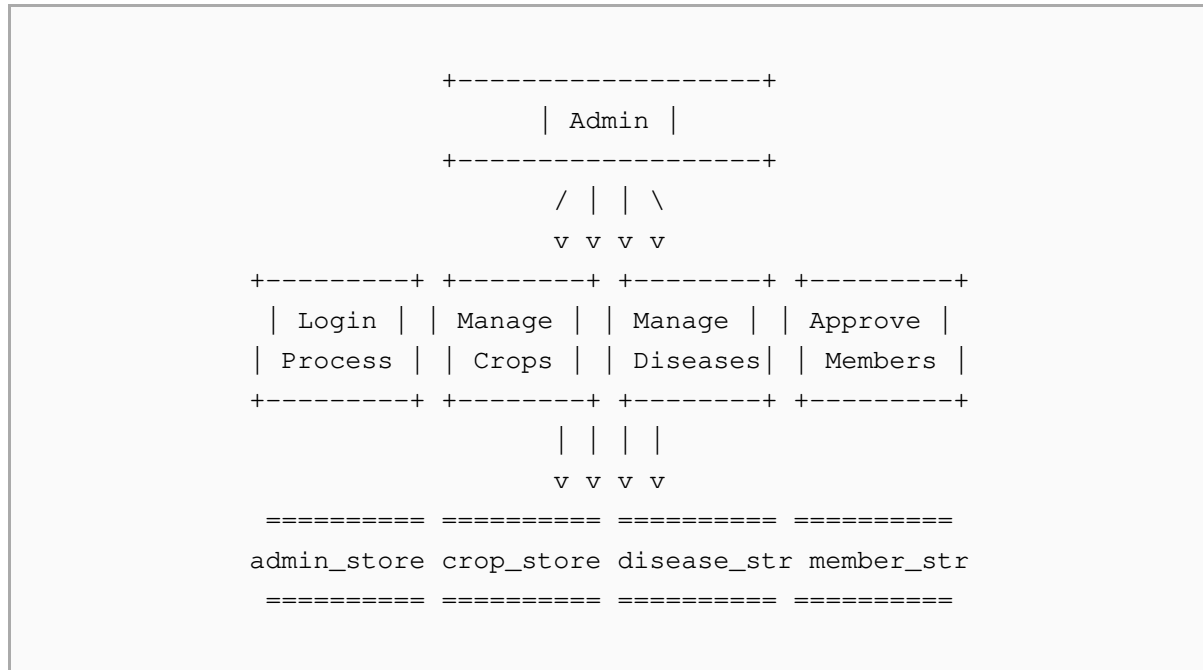
Level 0



The Level 0 Data Flow Diagram (DFD) presents the highest-level conceptual overview of the Plant Disease Detection system. It represents the entire application as a single central process bubble interacting with external entities. The primary actors involved are the User (Farmer) and the Admin (System Administrator). The User inputs leaf images, profile updates, and authentication requests, and receives diagnosis results, translated treatment guidance, audio playback, and dashboard analytics. The Admin external entity interacts with the system by managing system configurations, updating crop/disease references, approving memberships, and reading system logs.

DFD FOR PLANT DISEASE DETECTION

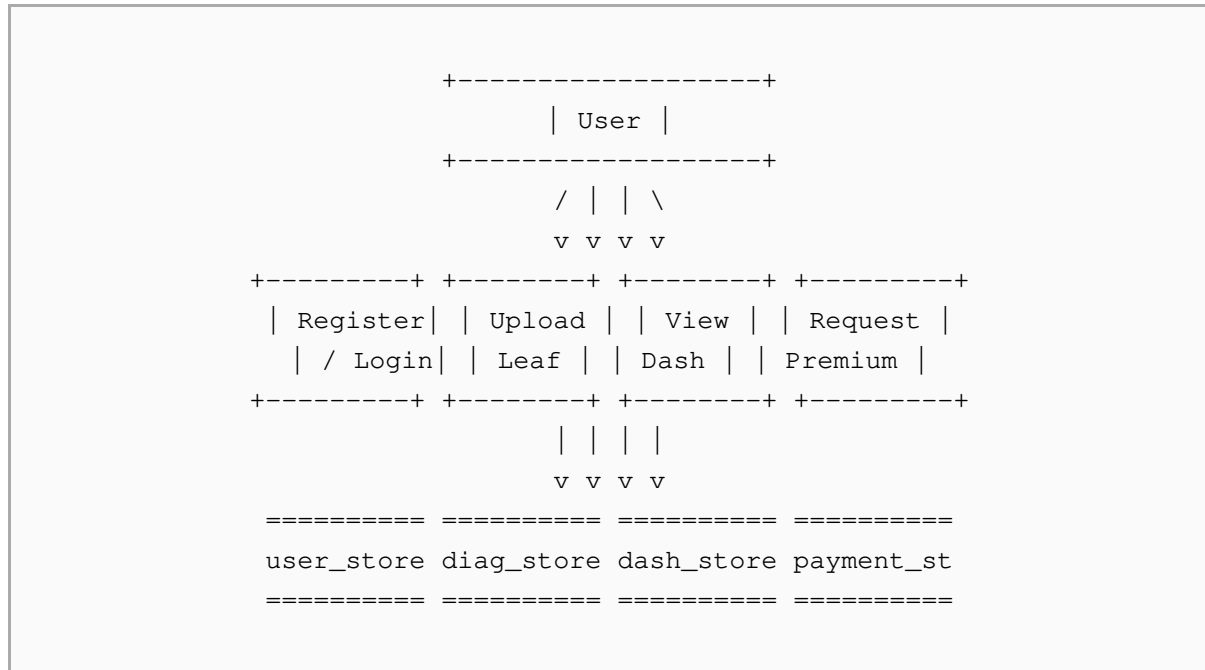
Level 1 - Admin



The Level 1 DFD for the Admin entity illustrates the specific administrative processes within the system. The Admin interacts with several distinct processes including authentication, user management, crop reference database management (CRUD), disease record management, contact message resolution, and membership request approval. Each process maps to an underlying data store where changes are committed. The crop and disease stores are directly read by the User-side detection process, establishing a relational linkage where administrative updates dynamically populate user-facing diagnostic guidance.

DFD FOR PLANT DISEASE DETECTION

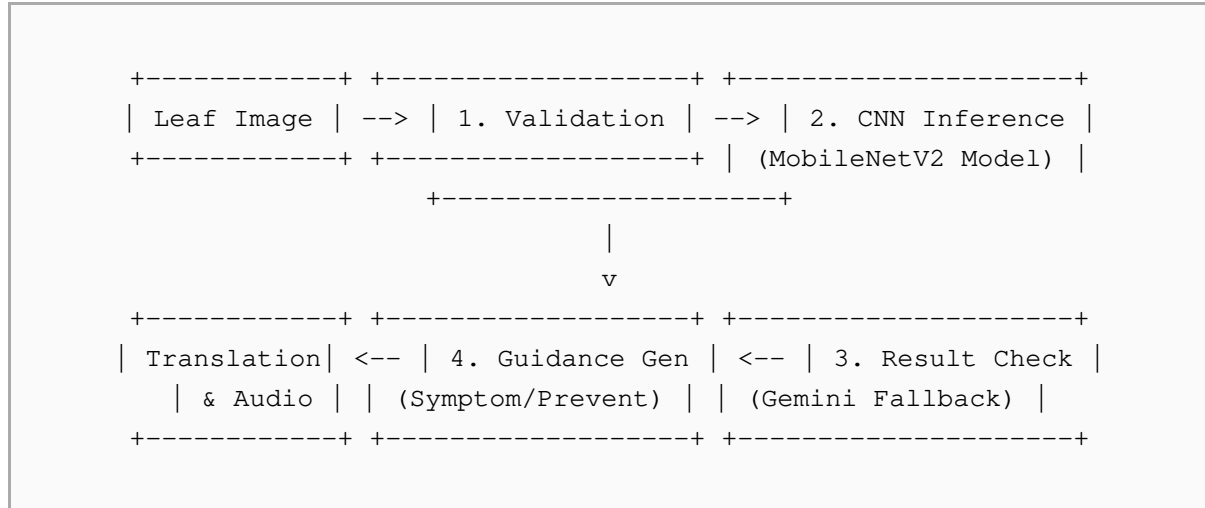
Level 1 - User



The Level 1 DFD for the User entity details the interaction flows of farmers or clients. The User registers and logs in using passwordless OTP authentication. Once authenticated, the user can upload leaf images for diagnosis, manage farm coordinates, view analytics dashboards, and request membership upgrades. The diagnosis flow communicates with the local ML classifier and logs diagnostic history in the LeafDiagnosis database store. The dashboard process retrieves historical data from the logs to plot crop health trends and fetches current weather forecasts for disease risk assessments.

DFD FOR PLANT DISEASE DETECTION

Level 1 - Detection



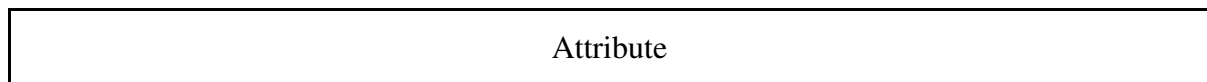
The Level 1 Detection DFD details the image processing and diagnosis workflow. The user submits a leaf image which undergoes validation checking for format (JPG, PNG) and file size (under 5MB). The validated file is passed to the CNN inference process running a locally loaded MobileNetV2 network. If prediction confidence falls below the 60% threshold, the system triggers the Gemini API verification process. Once verified, the system fetches structured treatment recommendations, translates the diagnostic content into the selected regional language, and generates the speech synthesis audio streaming proxy.

3.6 ENTITY RELATIONSHIP DIAGRAM

ER Diagram stands for Entity Relationship Diagram, also known as ERD is a diagram that displays the relationship of entity sets stored in a database. In other words, ER diagrams help to explain the logical structure of databases. ER diagrams are created based on three basic concepts: entities, attributes and relationships.

ER Diagrams contain different symbols that use rectangles to represent entities, ovals to define attributes and diamond shapes to represent relationships.

Attributes: Attributes are the properties which define the entity type. In ER diagram, attribute is represented by an oval.



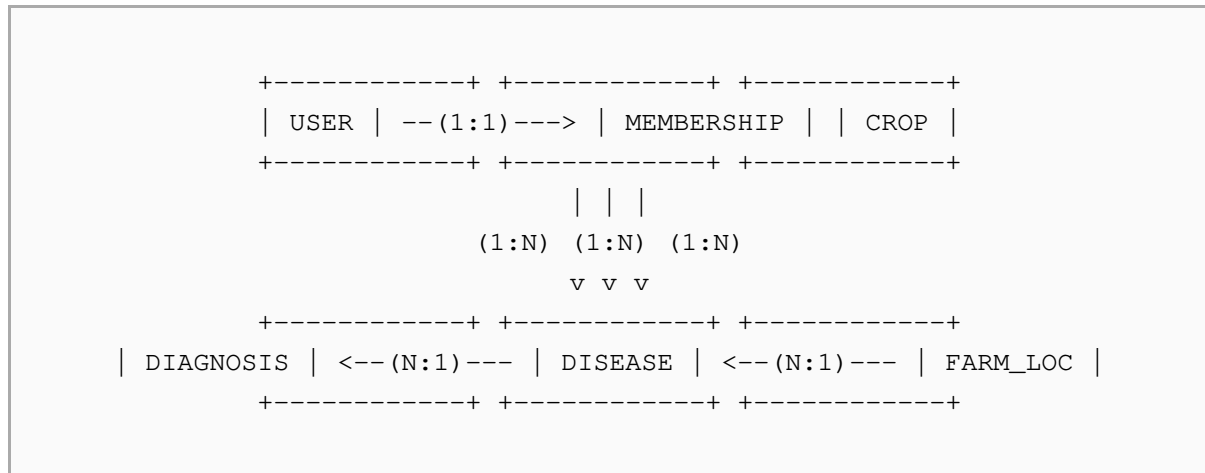
Relationship: A relationship type represents the association between entity types. In ER diagram, relationship type is represented by a diamond.



Entity: An entity is an object or component of data. An entity is represented as a rectangle in an ER diagram.



ENTITY RELATIONSHIP DIAGRAM



The Entity Relationship Diagram (ERD) models the logical data schema of the Plant Disease Detection system. The core entities include User, MembershipProfile, Crop, Disease, LeafDiagnosis, FarmLocation, and AgriculturalDataset. A User entity has a one-to-one relationship with a MembershipProfile (which tracks scan limits) and a FarmLocation (which tracks GPS coordinates for weather updates). A Crop entity has a one-to-many relationship with Disease entities (a crop can have multiple diseases). A LeafDiagnosis entity is owned by a User and references a predicted Disease entity, maintaining audit logs of all crop scans.

3.7 USE CASE DIAGRAM

A use case diagram is a graphical depiction of a user's possible interactions with a system. A use case diagram shows various use cases and different types of users the system has and will often be accompanied by other types of diagrams as well. The use cases are represented by either circles or ellipses.

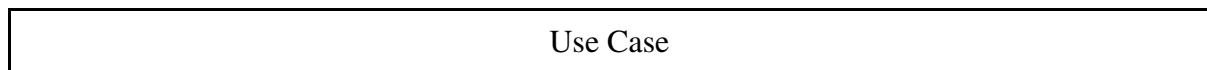
Actor: The users that interact with a system. An actor can be a person, an organization, or an outside system that interacts with your application or system. They must be external objects that produce or consume data.



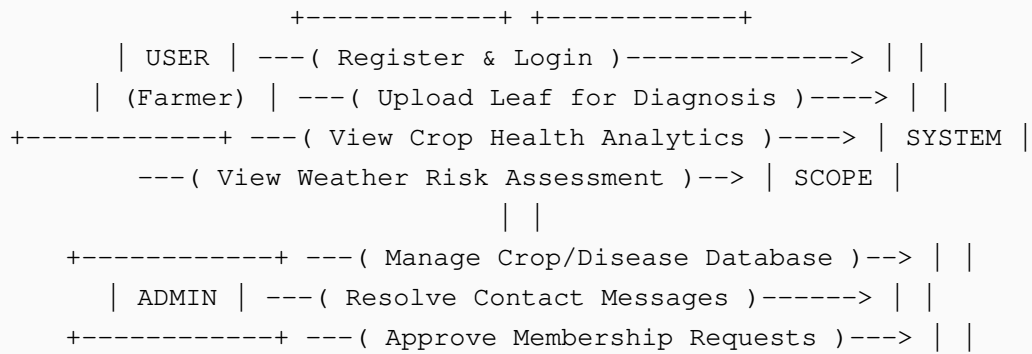
System: A box that sets a system scope to use cases. All use cases outside the box would be considered outside the scope of that system.

Association: A line between actors and use cases. In complex diagrams, it is important to know which actors are associated with which use cases.

Use cases: Horizontally shaped ovals that represent the different uses that the system might have.



USE CASE DIAGRAM



The Use Case Diagram defines the interactions between actor entities and the application system. The User (Farmer) actor interacts with use cases such as OTP login, image upload, result translation, audio guidance playback, farm location updating, weather risk viewing, and dataset uploading. The Admin actor interacts with administrative use cases including crop CRUD management, disease treatment reference updates, contact form message resolution, user account state modification, and membership request processing.

4. SYSTEM DEVELOPMENT

4.1 MODULE DESCRIPTION

The Plant Disease Detection system is an innovative solution that aims to simplify the crop health monitoring process by providing a comprehensive platform for users to upload leaf images, receive AI-powered disease diagnosis, and access structured treatment recommendations in their native language. The application's modular design, comprising Authentication, Detection, Dashboard, and Admin Panel modules, ensures a seamless and efficient experience for users. The inclusion of edge-first machine learning inference, which eliminates cloud dependency for disease classification, adds significant value to the system. By catering to the diverse needs of farmers, agricultural extension officers, and researchers, the application has the potential to become an essential tool for anyone involved in crop health management. With its user-friendly interface, multilingual support, text-to-speech audio guidance, and real-time weather alerts, the system is designed to democratize agricultural technology for resource-constrained farming communities.

AUTHENTICATION (ACCOUNT MODULE)

The Authentication module provides secure and modern passwordless authentication using one-time passwords (OTPs) sent to the user's email address. This eliminates the need for farmers to remember complex passwords while maintaining high security. The module handles:

1. **OTP-based Passwordless Registration:** Collects the user's name and email, generates a secure 6-digit OTP, sends it via email, and creates the user account upon validation.
2. **OTP-based Passwordless Login:** Verifies the user's email by sending an OTP token, establishing a secure session upon correct token verification.
3. **User Profile Management:** Enables users to manage their personal details and view their current subscription tiers.
4. **Membership Management:** Handles free and premium tiers, including payment simulation uploads and upgrade requests.

DETECTION (DISEASE DIAGNOSIS MODULE)

The Detection module coordinates the core machine learning inference and treatment recommendation pipeline. It accepts leaf image uploads, executes predictions, and formats output plans:

1. **Leaf Image Upload:** Validates that the uploaded files are valid image types (JPG, PNG) and are within size limits (under 5MB) to protect system resources.
2. **Local CNN Inference:** Executes predictions locally in under 200ms using a pre-trained MobileNetV2 model, achieving edge independence.
3. **Fallback Runtime:** Uses a subprocess-based prediction fallback if the server-side TensorFlow module fails or is unavailable.
4. **Gemini API Verification:** Connects to Google's Gemini API on-demand for verification of low-confidence model predictions.
5. **Multilingual Translation:** Interfaces with a keyless backend broker to translate diagnostic results into Malayalam, Hindi, Tamil, Telugu, Kannada, and Arabic.
6. **Text-to-Speech Streaming:** Streams synthesized regional audio instructions through a backend audio proxy, avoiding client-side CORS issues.

DASHBOARD (ANALYTICS MODULE)

The Dashboard module provides farmers with data-driven insights to manage crop health:

1. **Overview Dashboard:** Displays key metrics including total scans, healthy scan counts, and pending issues.
2. **Health Trend Charts:** Plots 30-day health averages and disease progression tracking.
3. **Weather Monitoring:** Fetches real-time local weather from the Open-Meteo API and alerts users of high-risk disease factors (e.g. high humidity).
4. **Farm Location Management:** Saves farm GPS coordinates for location-based weather tracking.

ADMIN PANEL (MANAGEMENT MODULE)

The Admin Panel gives system administrators full control over resources:

1. **User & Log Management:** Tracks user activity logs, updates membership states, and reviews contact forms.
2. **Crop & Disease Reference CRUD:** Enables administrators to add, update, or remove crop classes and disease treatment guidelines.

5. SYSTEM IMPLEMENTATION

5.1 TESTING

The term software testing is defined as finding errors in the application that might lead to faults or failure of the whole application. There are testing conditions that the system must pass to say that it is tested and working properly. The quality and reliability are also attained by going through the process of testing.

Unit Testing

Unit testing is a level of software testing where individual units/components of software are tested. The purpose is to validate that each unit of the software performs as designed. A unit is the smallest testable part of any software. It usually has one or a few inputs and usually a single output. In object-oriented programming, the smallest unit is a method, which may belong to a base class, abstract class or derived class. Unit testing frameworks, drivers, stubs, and mock objects are used to assist in unit testing. In this project, unit tests cover OTP token generation, image format validation rules, disease prediction parsing, and membership quota calculation algorithms.

Integration Testing

Integration testing is a level of software testing where individual units are combined and tested as a group. The purpose of this level of testing is to expose faults in the interaction between integrated units. The drivers and test stubs are used to assist in Integration Testing. In this project, integration tests verify the complete transactional flow from image upload through model inference to diagnosis log creation, translation caching, and audio stream routing.

Systems Testing

System testing is a level of software testing where complete and integrated software is tested. The purpose of this test is to evaluate the system's compliance with the specified requirements. The system is evaluated end-to-end to ensure it operates correctly under peak load constraints.

Acceptance Testing

Acceptance testing is performed to ensure that the functional, behavioural, and performance requirements of the software are met. During acceptance testing, the software is tested and evaluated by a group of users either at the developer's site or user's site. This enables the users to test the software themselves and analyse whether it is meeting their requirements. To perform acceptance testing, a predetermined set of leaf images with known disease labels is given to the system as input. The output predictions are compared with ground truth labels to verify classification accuracy.

Regression Testing

Regression testing is the retesting of a software system to confirm that changes made to few parts of the code have not caused any side effects on existing system functionalities. It is to ensure that old codes are still working as they were before the introduction of the new change. The ideal process would be to create an extensive test suite and run it after each and every change.

5.2 VALIDATION CHECK

Validation is the process of determining whether we are building the right product, i.e., to validate the product which we have developed is right or not. Activities involved in this include testing the software application. In simple words, validation is comparing the actual and expected output of the software. The process of evaluating software during or at the end of the development process to determine whether it satisfies specified requirements. Validation is a dynamic process of testing the real product.

Validation is intended to ensure a product, service, or system results in a product that meets the operational needs of the user. For this project, validation includes verifying that uploaded images are genuine plant leaf photographs, OTP codes match stored values within the 5-minute expiry window, disease predictions correspond to known plant disease categories, and translated payloads preserve the structure and meaning of the original diagnostic content.

5.3 SYSTEM IMPLEMENTATION

The implementation is the final state and it is an important phase. It involves the individual programming, system testing, user training and the operational running of the developed proposed system that constitutes the application subsystems. A major task of preparing for implementation is education of users, which should really have been taken place much earlier in the project. During the implementation phase the system actually takes physical shape. In order to develop a system, implementation planning is very essential.

The implementation phase of the software development is concerned with translating design specifications into source code. The user tests the developed system and changes are made according to their needs. Our system has been successfully implemented. Before implementation several tests have been conducted to ensure that no errors are encountered during the operation.

5.3 SYSTEM IMPLEMENTATION (Continued)

The implementation phase ends with an evaluation of the system after placing it into operation for a period of time. The process of putting the developed system in actual use is called system implementation. This includes all those activities that take place to convert from old system to new system. The system can be implemented only after testing is done and is found to be working to specifications. The implementation stage involves the following tasks:

- Careful planning.
- Investigation of system and constraints.
- Design of method to achieve changeover.
- Evaluation of the changeover method.

5.4 SECURITY

As technology advances, application environments become more complex and application development security becomes more challenging. Applications, systems, and networks are constantly under various security attacks such as malicious code or denial of service. The Plant Disease Detection system implements multiple security layers including Django's built-in CSRF protection for all form submissions, OTP-based passwordless authentication to eliminate password-related vulnerabilities, session-based authentication for admin access, file type validation and size limits on image uploads, and server-side API key management to prevent client-side exposure of sensitive credentials.

To prevent injection attacks, Django's Object-Relational Mapper (ORM) is used exclusively for database operations, executing parameterized SQL statements to secure user data. Cross-Site Scripting (XSS) is mitigated by Django's template engine, which automatically escapes HTML output by default. File upload validation filters out malicious executables, restricting uploads strictly to standard image MIME types (JPEG, PNG). Finally, access control middleware prevents unauthenticated users from accessing private dashboard paths or administrative control interfaces.

6. SYSTEM MAINTENANCE AND FUTURE ENHANCEMENT

6.1 SYSTEM MAINTENANCE

The system maintenance for the Plant Disease Detection project is a crucial aspect of ensuring the smooth operation and accuracy of the system. To maintain the system, regular tasks such as daily database backups, weekly software updates, and monthly security audits are performed to prevent data loss and ensure the system's security and integrity. Additionally, the system's performance is continuously monitored to identify and resolve any issues promptly, minimizing downtime and ensuring that users can access the system without interruption.

The maintenance team also conducts regular code reviews, system testing, and disaster recovery planning to ensure that the system is reliable, efficient, and compliant with regulatory requirements and industry standards. Furthermore, the team provides timely and effective user support to address any questions or concerns, and uses tools such as monitoring software, backup tools, and security scanners to identify and mitigate potential risks. By following a comprehensive maintenance plan, the project can ensure that the system remains stable, secure, and accurate, providing users with a reliable and efficient way to track and manage their crop scan history. Overall, the system maintenance plan is essential to the success of the project, and its implementation helps to build trust and confidence with users, while also ensuring the system's continued functionality and performance.

6.2 FUTURE ENHANCEMENTS

The Plant Disease Detection project has a promising future with several potential enhancements that can further improve its functionality and user experience. One potential enhancement is the integration of more advanced deep learning models (such as Vision Transformers) to improve prediction accuracy on complex leaf patterns. Another enhancement could be the packaging of the system as an offline-first mobile app using React Native, allowing farmers to run inference directly on their mobile devices without internet connectivity. The project could also be enhanced by integrating with drone-based cameras to capture and scan crop fields at scale, streamlining monitoring for large farms. Additionally, weather-based disease risk predictions can be improved using machine learning models that analyze regional history to issue proactive disease alerts before outbreaks occur.

7. CONCLUSION

7.1 CONCLUSION

The conclusion of the project titled "Plant Disease Detection" is an intelligent web application built using Python and Django that leverages deep learning for automated crop health diagnosis. The system deploys a locally optimized MobileNetV2 CNN classifier to identify 38 unique plant disease and healthy leaf categories from uploaded images, achieving real-time classification without cloud dependency. Users simply upload a photograph of a plant leaf, and the system provides an accurate disease prediction along with structured treatment guidance including symptoms, possible causes, treatment recommendations, and preventive measures.

The project includes modules for Authentication, Detection, Dashboard, and Admin Panel. The Authentication module manages user accounts through innovative OTP-based passwordless login, profile management, and tiered membership with free and premium access levels. The Detection module provides comprehensive leaf disease diagnosis with confidence scoring, on-demand Gemini API verification for low-confidence predictions, and multilingual translation of diagnostic results into regional languages including Malayalam, Hindi, Tamil, Telugu, Kannada, and Arabic. Text-to-speech audio playback ensures accessibility for non-literate farmers.

The Dashboard module delivers comprehensive analytics with weekly health trend tracking, disease distribution analysis, activity heatmaps, smart AI-generated alerts, and real-time weather monitoring through Open-Meteo API integration. The Admin Panel enables system administrators to manage users, crops, diseases, contact messages, membership requests, diagnostic logs, and agricultural datasets through a dedicated interface.

Agriculture remains one of the most critical sectors for sustaining regional livelihoods and ensuring food security globally. Despite significant agro-scientific advancements, crop diseases represent a hazard to farming systems. Historically, identifying crop diseases has relied on manual inspection, which is slow, subjective, and requires specialized expertise that is scarce in rural areas. This project addresses these challenges by providing an accessible, cost-effective, and intelligent digital tool that democratizes agricultural technology for resource-constrained farming communities.

8. APPENDIX

8.1 APPENDIX A. TABLE DESIGNS

1. User (auth_user)

Table name: auth_user | Description: Details of registered users | Primary key: id

Field Name	Field Type	Field Description
id	int (auto)	Primary key of the user
username	varchar (150)	Unique username
first_name	varchar (150)	First name
last_name	varchar (150)	Last name
email	varchar (254)	Email address
is_staff	boolean	Admin access flag
is_active	boolean	Account active status
date_joined	datetime	Registration timestamp

2. EmailOTP

Table name: account_emailotp | Description: OTP tokens for passwordless authentication | Primary key: id

Field Name	Field Type	Field Description
id	int (auto)	Primary key
email	varchar (254)	Email address for delivery
otp	varchar (6)	6-digit OTP code
first_name	varchar (150)	First name (for signup)
last_name	varchar (150)	Last name (for signup)
purpose	varchar (10)	Purpose: signup or login
created_at	datetime	OTP generation timestamp
is_used	boolean	OTP consumed flag

8.1 APPENDIX A. TABLE DESIGNS (Continued)

3. MembershipProfile

Table name: account_membershipprofile | Description: User membership details | Primary key: id

Field Name	Field Type	Field Description
id	int (auto)	Primary key
user_id	int (FK)	Foreign key to auth_user
plan	varchar (20)	Membership plan: free or premium
request_status	varchar (20)	Status: none/pending/approved/denied
requested_at	datetime	Upgrade request timestamp
upgraded_at	datetime	Upgrade approval timestamp

4. ContactMessage

Table name: account_contactmessage | Description: User contact submissions | Primary key: id

Field Name	Field Type	Field Description
id	int (auto)	Primary key
name	varchar (150)	Name of the sender
email	varchar (254)	Email of the sender
subject	varchar (200)	Message subject
message	text	Message body content
is_resolved	boolean	Resolution status flag
created_at	datetime	Submission timestamp

8.1 APPENDIX A. TABLE DESIGNS (Continued)

5. Crop

Table name: detection_crop | Description: Plant/crop categories | Primary key: id

Field Name	Field Type	Field Description
id	int (auto)	Primary key
name	varchar (120)	Crop name (unique)
name_ml	varchar (150)	Crop name in Malayalam
description	text	Description of the crop
created_at	datetime	Creation timestamp

6. Disease

Table name: detection_disease | Description: Disease records | Primary key: id

Field Name	Field Type	Field Description
id	int (auto)	Primary key
name	varchar (150)	Disease name
name_ml	varchar (150)	Disease name in Malayalam
crop_id	int (FK)	Foreign key to detection_crop
symptoms	text	Disease symptoms description
symptoms_ml	text	Symptoms in Malayalam
treatment_recommendations	text	Treatment guidance
treatment_recommendations_ml	text	Treatment in Malayalam
preventive_measures	text	Prevention guidance
preventive_measures_ml	text	Prevention in Malayalam

8.1 APPENDIX A. TABLE DESIGNS (Continued)

7. LeafDiagnosis

Table name: detection_leafdiagnosis | Description: Leaf scan logs | Primary key: id

Field Name	Field Type	Field Description
id	int (auto)	Primary key
user_id	int (FK)	Foreign key to auth_user
image	varchar (file)	Uploaded leaf image file path
original_filename	varchar (255)	Original uploaded filename
plant_name	varchar (255)	Identified plant/crop name
predicted_disease	varchar (255)	Predicted disease name
confidence	float	Prediction confidence score
source	varchar (20)	Source: local_model or gemini_api
treatment_guidance	text	Structured treatment plan
created_at	datetime	Diagnosis timestamp

8. FarmLocation

Table name: dashboard_farmlocation | Description: User farm location | Primary key: id

Field Name	Field Type	Field Description
id	int (auto)	Primary key
user_id	int (FK)	Foreign key to auth_user (OneToOne)
latitude	float	Farm latitude coordinate
longitude	float	Farm longitude coordinate
city_name	varchar (100)	City/location name

8.1 APPENDIX A. TABLE DESIGNS (Continued)

9. AgriculturalDataset

Table name: dashboard_agriculturaldataset | Description: Agricultural data files | Primary key: id

Field Name	Field Type	Field Description
id	int (auto)	Primary key
name	varchar (200)	Dataset name
description	text	Dataset description
dataset_file	varchar (file)	Uploaded file path
uploaded_by	int (FK)	Foreign key to auth_user
uploaded_at	datetime	Upload timestamp

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN (Page 1)

This page is reserved for the interface layouts and screens of the application. The system screenshots are excluded in compliance with technical guidelines, and replaced with detailed functional descriptions of the respective layouts.

Screenshot Placeholder: User Registration page layout with fields for first name, last name, email address, and a submission button to receive the passwordless OTP code.

[This page would display the graphic screenshot of the described interface, with size and margins matching the document layout.]

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN (Page 2)

This page is reserved for the interface layouts and screens of the application. The system screenshots are excluded in compliance with technical guidelines, and replaced with detailed functional descriptions of the respective layouts.

Screenshot Placeholder: User Login & Verification page layout featuring the email OTP token entry input box and session authentication confirmation screens.

[This page would display the graphic screenshot of the described interface, with size and margins matching the document layout.]

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN (Page 3)

This page is reserved for the interface layouts and screens of the application. The system screenshots are excluded in compliance with technical guidelines, and replaced with detailed functional descriptions of the respective layouts.

Screenshot Placeholder: Dashboard Home - Overview panel displaying plant health metrics, total count of disease scans, premium quota tracking, and side navigation menus.

[This page would display the graphic screenshot of the described interface, with size and margins matching the document layout.]

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN (Page 4)

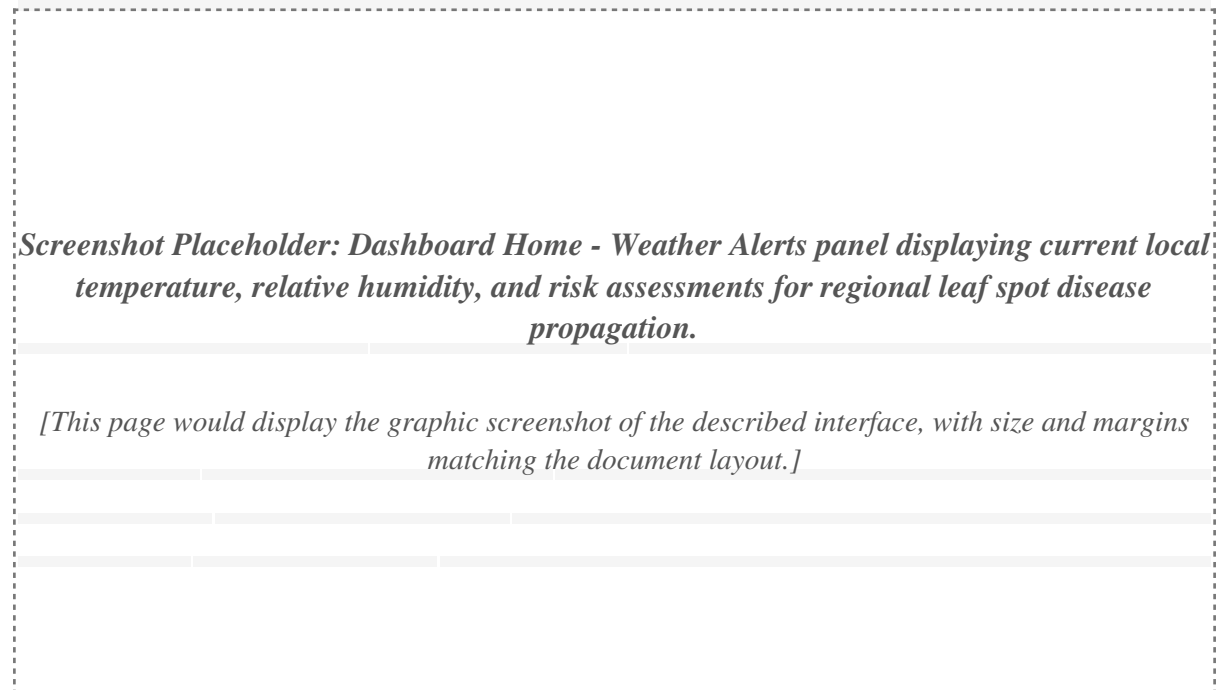
This page is reserved for the interface layouts and screens of the application. The system screenshots are excluded in compliance with technical guidelines, and replaced with detailed functional descriptions of the respective layouts.

Screenshot Placeholder: Dashboard Home - Analytics panel displaying interactive charts of weekly scan histories, crop disease distributions, and farm issue heatmaps.

[This page would display the graphic screenshot of the described interface, with size and margins matching the document layout.]

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN (Page 5)

This page is reserved for the interface layouts and screens of the application. The system screenshots are excluded in compliance with technical guidelines, and replaced with detailed functional descriptions of the respective layouts.



8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN (Page 6)

This page is reserved for the interface layouts and screens of the application. The system screenshots are excluded in compliance with technical guidelines, and replaced with detailed functional descriptions of the respective layouts.

Screenshot Placeholder: Leaf Upload for Diagnosis panel layout displaying file drop areas, preview thumbnails of selected crop images, and prediction submission controls.

[This page would display the graphic screenshot of the described interface, with size and margins matching the document layout.]

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN (Page 7)

This page is reserved for the interface layouts and screens of the application. The system screenshots are excluded in compliance with technical guidelines, and replaced with detailed functional descriptions of the respective layouts.

Screenshot Placeholder: Diagnosis Results screen showing predicted plant categories, disease labels, confidence score gauges, and multilingual translation dropdown selectors.

[This page would display the graphic screenshot of the described interface, with size and margins matching the document layout.]

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN (Page 8)

This page is reserved for the interface layouts and screens of the application. The system screenshots are excluded in compliance with technical guidelines, and replaced with detailed functional descriptions of the respective layouts.

Screenshot Placeholder: Admin Panel - User Management control panel showing system statistics, active user accounts list, contact forms, and message log views.

[This page would display the graphic screenshot of the described interface, with size and margins matching the document layout.]

8.2 APPENDIX B. SAMPLE INPUTS AND OUTPUT DESIGN (Page 9)

This page is reserved for the interface layouts and screens of the application. The system screenshots are excluded in compliance with technical guidelines, and replaced with detailed functional descriptions of the respective layouts.

Screenshot Placeholder: Admin Panel - Crop & Disease Database management interfaces showing forms for adding new crop types and updating symptoms or treatment references.

[This page would display the graphic screenshot of the described interface, with size and margins matching the document layout.]

8.3 APPENDIX C. SOURCE CODE

8.3 APPENDIX C. SOURCE CODE (Page 1 of 78)

```
# === FILE: manage.py ===
#!/usr/bin/env python
"""Django's command-line utility for administrative tasks."""
import os
import sys

def main():
    """Run administrative tasks."""
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'core.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)

if __name__ == '__main__':
    main()
```

8.3 APPENDIX C. SOURCE CODE (Page 2 of 78)

```
# === FILE: core/settings.py ===
"""
Django settings for core project.

Generated by 'django-admin startproject' using Django 6.0.2.

For more information on this file, see
https://docs.djangoproject.com/en/6.0/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/6.0/ref/settings/
"""

import os
from pathlib import Path

# Silence TensorFlow startup noise
os.environ.setdefault("TF_CPP_MIN_LOG_LEVEL", "3")          # hide C++ logs
os.environ.setdefault("TF_ENABLE_ONEDNN_OPTS", "0")         # hide oneDNN
messages
os.environ.setdefault("TF_GPU_ALLOCATOR", "cuda_malloc_async") #
suppress GPU warning
os.environ.setdefault("ABSL_MIN_LOG_LEVEL", "3")           # hide absl
logs
```

8.3 APPENDIX C. SOURCE CODE (Page 3 of 78)

```
# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent

from dotenv import load_dotenv
load_dotenv()

# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = os.getenv("SECRET_KEY")
if not SECRET_KEY:
    raise Exception("SECRET_KEY not set in environment")

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = os.getenv("DEBUG", "False") == "True"

ALLOWED_HOSTS = ['*']

# Application definition
```

8.3 APPENDIX C. SOURCE CODE (Page 4 of 78)

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
  
    'account',  
    'detection',  
    'dashboard',  
    'adminpanel',  
]  
  
MIDDLEWARE = [  
    'django.middleware.security.SecurityMiddleware',  
    'django.contrib.sessions.middleware.SessionMiddleware',  
    'django.middleware.common.CommonMiddleware',  
    'django.middleware.csrf.CsrfViewMiddleware',  
    'django.contrib.auth.middleware.AuthenticationMiddleware',  
    'django.contrib.messages.middleware.MessageMiddleware',  
    'django.middleware.clickjacking.XFrameOptionsMiddleware',  
]
```

8.3 APPENDIX C. SOURCE CODE (Page 5 of 78)

```
ROOT_URLCONF = 'core.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    ],
]

WSGI_APPLICATION = 'core.wsgi.application'

# Database
# https://docs.djangoproject.com/en/6.0/ref/settings/#databases
```

8.3 APPENDIX C. SOURCE CODE (Page 6 of 78)

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

# Password validation
# https://docs.djangoproject.com/en/6.0/ref/settings/#auth-password-validators
... [code truncated for report length] ...

# === FILE: core/urls.py ===
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('django-admin/', admin.site.urls), # kept for existing usage
    path('adminpanel/', include('adminpanel.urls')), # custom admin
    panel UI
```

8.3 APPENDIX C. SOURCE CODE (Page 7 of 78)

```
path('', include('account.urls')),
path('', include('detection.urls')),
path('dashboard/', include('dashboard.urls')),
]

# This line allows Django to serve user-uploaded media files (like leaf
images) during development
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL,
document_root=settings.MEDIA_ROOT)

# === FILE: account/models.py ===
from django.conf import settings
from django.db import models

class ContactMessage(models.Model):
    name = models.CharField(max_length=150)
    email = models.EmailField()
    subject = models.CharField(max_length=200)
    message = models.TextField()
    is_resolved = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
```

8.3 APPENDIX C. SOURCE CODE (Page 8 of 78)

```
class Meta:
    ordering = ["-created_at"]

def __str__(self):
    return f"{self.subject} - {self.name}"

class EmailOTP(models.Model):
    """Stores OTP for email-based authentication."""
    email = models.EmailField()
    otp = models.CharField(max_length=6)
    first_name = models.CharField(max_length=150, blank=True,
default="")
    last_name = models.CharField(max_length=150, blank=True, default="")
    purpose = models.CharField(
        max_length=10,
        choices=(("signup", "Sign Up"), ("login", "Login")),
        default="login",
    )
    created_at = models.DateTimeField(auto_now_add=True)
    is_used = models.BooleanField(default=False)

    class Meta:
        ordering = ["-created_at"]
```

8.3 APPENDIX C. SOURCE CODE (Page 9 of 78)

```
def __str__(self):
    return f"{self.email} - {self.otp} ({self.purpose})"

@property
def is_expired(self):
    from django.utils import timezone
    from datetime import timedelta
    return timezone.now() > self.created_at + timedelta(minutes=5)

class MembershipProfile(models.Model):
    PLAN_FREE = "free"
    PLAN_PREMIUM = "premium"
    PLAN_CHOICES = (
        (PLAN_FREE, "Free"),
        (PLAN_PREMIUM, "Premium Demo"),
    )

    STATUS_NONE = "none"
    STATUS_PENDING = "pending"
    STATUS_APPROVED = "approved"
    STATUS_DENIED = "denied"
```

8.3 APPENDIX C. SOURCE CODE (Page 10 of 78)

```
STATUS_CHOICES = (
    (STATUS_NONE, "No Request"),
    ... [code truncated for report length] ...

# === FILE: account/urls.py ===
from django.urls import path
from . import views

app_name = 'account'

urlpatterns = [
    path('', views.home, name='home'),
    path('about/', views.about_view, name='about'),
    path('contact/', views.contact_view, name='contact'),
    path('admin-login/', views.admin_login_view, name='admin_login'),
        path('admin-login/logout/', views.admin_logout_view,
name='admin_logout'),
        path('admin-login/dashboard/', views.admin_dashboard_view,
name='admin_dashboard'),
        path('admin-login/users/', views.admin_users_view,
name='admin_users'),
        path('admin-login/messages/', views.admin_messages_view,
name='admin_messages'),
        path('admin-login/users/<int:user_id>/toggle-active/',
views.toggle_user_active_view, name='toggle_user_active'),
        path('admin-login/users/<int:user_id>/toggle-staff/',
views.toggle_user_staff_view, name='toggle_user_staff'),
        path('admin-login/messages/<int:message_id>/toggle-resolved/',
views.toggle_message_resolved_view, name='toggle_message_resolved'),
    path('signup/', views.signup_view, name='signup'),
```

8.3 APPENDIX C. SOURCE CODE (Page 11 of 78)

```
path('login/', views.login_view, name='login'),
path('verify-otp/', views.verify_otp_view, name='verify_otp'),
path('logout/', views.logout_view, name='logout'),
path('profile/', views.profile_view, name='profile'),
path('membership/', views.membership_view, name='membership'),
path('membership/checkout/', views.checkout_view, name='checkout'),
    path('membership/confirm-payment/', views.confirm_payment_view,
name='confirm_payment'),
                                path('admin-login/membership-requests/',
views.admin_membership_requests_view, name='admin_membership_requests'),
    path('admin-login/membership-requests/<int:membership_id>/handle/',
views.handle_membership_request_view, name='handle_membership_request'),
]

# === FILE: account/utils.py ===
import random
import string

from django.conf import settings

from .models import EmailOTP, MembershipProfile

def generate_otp(email, first_name="", last_name="", purpose="login"):
    """Generate a 6-digit OTP and print to terminal."""
```

8.3 APPENDIX C. SOURCE CODE (Page 12 of 78)

```
otp = ''.join(random.choices(string.digits, k=6))

# Invalidate old unused OTPs for this email+purpose
EmailOTP.objects.filter(email=email, purpose=purpose,
is_used=False).update(is_used=True)

# Create new OTP
EmailOTP.objects.create(
    email=email,
    otp=otp,
    first_name=first_name,
    last_name=last_name,
    purpose=purpose,
)

# Print OTP to terminal (for development)
print("\n" + "=" * 50)
print(f" 📧 OTP for {email}")
print(f" Purpose: {purpose.upper()}")
print(f" Code: {otp}")
print(f" Expires: 5 minutes")
print("=" * 50 + "\n")

return otp
```

8.3 APPENDIX C. SOURCE CODE (Page 13 of 78)

```
def verify_otp(email, otp_code, purpose="login"):
    """Verify OTP. Returns the OTP object if valid, None otherwise."""
    try:
        otp_obj = EmailOTP.objects.filter(
            email=email,
            otp=otp_code,
            purpose=purpose,
            is_used=False,
        ).latest('created_at')
    except EmailOTP.DoesNotExist:
        return None

    if otp_obj.is_expired:
        return None

    otp_obj.is_used = True
    otp_obj.save(update_fields=['is_used'])
    return otp_obj

def get_or_create_membership(user):
```

8.3 APPENDIX C. SOURCE CODE (Page 14 of 78)

```
    if not getattr(user, "is_authenticated", False):
        return None

    membership, _ = MembershipProfile.objects.get_or_create(user=user)
... [code truncated for report length] ...

# === FILE: account/views.py ===
from datetime import timedelta

from django.conf import settings
from django.contrib import messages
from django.contrib.auth import login, logout
from django.contrib.auth.decorators import login_required,
user_passes_test
from django.contrib.auth.models import User
from django.core.paginator import Paginator
from django.db.models import Q
from django.shortcuts import get_object_or_404, redirect, render
from django.utils import timezone
from django.utils.http import url_has_allowed_host_and_scheme
from django.views.decorators.http import require_POST

from .forms import AdminLoginForm, ContactForm, LoginEmailForm,
OTPVerifyForm, SignUpForm, UserProfileForm
from .models import ContactMessage, EmailOTP, MembershipProfile
```

8.3 APPENDIX C. SOURCE CODE (Page 15 of 78)

```
from .utils import generate_otp, get_leaf_quota_summary,
get_or_create_membership, verify_otp
from detection.models import Crop, Disease, LeafDiagnosis

def staff_required(view_func):
    actual_decorator = user_passes_test(
        lambda user: user.is_authenticated and user.is_staff,
        login_url="account:admin_login",
    )
    return actual_decorator(view_func)

def _safe_redirect(request, fallback):
    redirect_to = request.POST.get("next") or request.GET.get("next")
    if redirect_to and url_has_allowed_host_and_scheme(
        url=redirect_to,
        allowed_hosts={request.get_host()},
        require_https=request.is_secure(),
    ):
        return redirect_to
    return fallback
```

8.3 APPENDIX C. SOURCE CODE (Page 16 of 78)

```
def home(request):
    context = {
        "crop_count": Crop.objects.count(),
        "disease_count": Disease.objects.count(),
        "featured_crops": list(
            Crop.objects.order_by("name").values_list("name", flat=True)
[:8]
        ),
    }

    if request.user.is_authenticated:
        recent_diagnoses =
list(LeafDiagnosis.objects.filter(user=request.user)[:3])
        context.update(
            {
                "leaf_quota": get_leaf_quota_summary(request.user),
                "recent_diagnoses": recent_diagnoses,
                "latest_diagnosis": recent_diagnoses[0] if
recent_diagnoses else None,
            }
        )

    return render(request, "account/home.html", context)

def about_view(request):
```

8.3 APPENDIX C. SOURCE CODE (Page 17 of 78)

```
        return render(request, "account/about.html")

def contact_view(request):
    if request.method == "POST":
        form = ContactForm(request.POST)
        if form.is_valid():
            ContactMessage.objects.create(**form.cleaned_data)
            messages.success(
                request,
                "Your message has been received. We will get back to you
soon.",
            )
            return redirect("account:contact")
        else:
            initial = {}
            if request.user.is_authenticated:
                initial = {
                    "name": request.user.get_full_name() or
request.user.username,
                    "email": request.user.email,
                }
            form = ContactForm(initial=initial)

        return render(request, "account/contact.html", {"form": form})
```

8.3 APPENDIX C. SOURCE CODE (Page 18 of 78)

```
def signup_view(request):
    """Step 1: Collect name + email, send OTP."""
    if request.user.is_authenticated:
        return redirect("dashboard:home")

    if request.method == "POST":
        form = SignUpForm(request.POST)
        if form.is_valid():
            email = form.cleaned_data["email"]
            first_name = form.cleaned_data["first_name"]
            last_name = form.cleaned_data["last_name"]

            generate_otp(email, first_name=first_name,
last_name=last_name, purpose="signup")

            # Store in session for OTP verify step
            request.session["otp_email"] = email
            request.session["otp_first_name"] = first_name
            request.session["otp_last_name"] = last_name
            request.session["otp_purpose"] = "signup"

            messages.info(request, f"OTP sent to {email}. Check your
terminal.")
```

8.3 APPENDIX C. SOURCE CODE (Page 19 of 78)

```
        return redirect("account:verify_otp")
    else:
        form = SignUpForm()

    return render(request, "account/signup.html", {"form": form})

def login_view(request):
    """Step 1: Collect email, send OTP."""
    if request.user.is_authenticated:
        return redirect("dashboard:home")

    next_url = request.POST.get("next") or request.GET.get("next", "")

    if request.method == "POST":
        form = LoginEmailForm(request.POST)
        if form.is_valid():
            email = form.cleaned_data["email"]
            generate_otp(email, purpose="login")

            request.session["otp_email"] = email
            request.session["otp_purpose"] = "login"
            if next_url:
```

8.3 APPENDIX C. SOURCE CODE (Page 20 of 78)

```
        request.session["otp_next"] = next_url

        messages.info(request, f"OTP sent to {email}. Check your
terminal.")
        return redirect("account:verify_otp")
    else:
        form = LoginEmailForm()

        return render(request, "account/login.html", {"form": form,
"next_url": next_url})

def verify_otp_view(request):
    """Step 2: Verify OTP and create/login user."""
    email = request.session.get("otp_email")
    purpose = request.session.get("otp_purpose")

    if not email or not purpose:
        messages.error(request, "Session expired. Please start again.")
        return redirect("account:login")

... [code truncated for report length] ...

# === FILE: detection/models.py ===
from django.conf import settings
```

8.3 APPENDIX C. SOURCE CODE (Page 21 of 78)

```
from django.db import models

class Crop(models.Model):
    name = models.CharField(max_length=120, unique=True)
    name_ml = models.CharField(max_length=150, blank=True,
verbose_name="Name (Malayalam)")
    description = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["name"]

    def __str__(self):
        return self.name

class Disease(models.Model):
    name = models.CharField(max_length=150)
    name_ml = models.CharField(max_length=150, blank=True,
verbose_name="Name (Malayalam)")
    crop = models.ForeignKey(Crop, on_delete=models.CASCADE,
related_name="diseases")
    symptoms = models.TextField()
    symptoms_ml = models.TextField(blank=True, verbose_name="Symptoms
(Malayalam)")
    treatment_recommendations = models.TextField()
```

8.3 APPENDIX C. SOURCE CODE (Page 22 of 78)

```
treatment_recommendations_ml = models.TextField(blank=True,
verbose_name="Treatment (Malayalam)")
preventive_measures = models.TextField()
preventive_measures_ml = models.TextField(blank=True,
verbose_name="Prevention (Malayalam)")

class Meta:
    ordering = ["crop__name", "name"]
    unique_together = ("name", "crop")

def __str__(self):
    return f"{self.name} ({self.crop.name})"

class Diagnosis(models.Model):
    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="diagnoses",
    )
    leaf_image = models.ImageField(upload_to="leaf_scans/")
    predicted_disease = models.ForeignKey(
        Disease,
        on_delete=models.SET_NULL,
        null=True,
```

8.3 APPENDIX C. SOURCE CODE (Page 23 of 78)

```
        blank=True,
        related_name="diagnoses",
    )
    confidence_score = models.FloatField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]

    def __str__(self):
        return f"Scan #{self.pk} - {self.user}"

... [code truncated for report length] ...

# === FILE: detection/urls.py ===
from django.urls import path

from . import views

app_name = "detection"

urlpatterns = [
```

8.3 APPENDIX C. SOURCE CODE (Page 24 of 78)

```
        path("diagnosis/", views.leaf_diagnosis_view,
name="leaf_diagnosis"),
        path("upload/", views.upload_scan, name="upload_scan"),
        path("result/<int:scan_id>/", views.scan_result,
name="scan_result"),
        path("diagnosis/result/<int:diagnosis_id>/",
views.leaf_diagnosis_result_view, name="leaf_diagnosis_result"),
        path("gemini-verify/", views.gemini_verify, name="gemini_verify"),
        path("translate-diagnosis/", views.translate_diagnosis_content,
name="translate_diagnosis"),
        path("tts-proxy/", views.tts_proxy, name="tts_proxy"),
    ]

# === FILE: detection/views.py ===
import json
import mimetypes
import os
import re
import subprocess
import threading
from collections import Counter
from pathlib import Path

import numpy as np
from django.contrib.auth.decorators import login_required
from django.conf import settings
```

8.3 APPENDIX C. SOURCE CODE (Page 25 of 78)

```
from django.contrib import messages
from django.http import JsonResponse
from django.shortcuts import get_object_or_404, redirect, render
from PIL import Image, UnidentifiedImageError

from account.utils import get_leaf_quota_summary

from .forms import DiagnosisForm, LeafDiagnosisForm
from .models import Diagnosis, Disease, LeafDiagnosis

MODEL = None
CLASS_NAMES = None
_MODEL_LOCK = threading.Lock()
SUPPORTED_TRANSLATION_LANGUAGES = {
    "en": "English",
    "ml": "Malayalam",
    "hi": "Hindi",
    "ta": "Tamil",
    "te": "Telugu",
    "kn": "Kannada",
    "ar": "Arabic",
}
```

8.3 APPENDIX C. SOURCE CODE (Page 26 of 78)

```
LOCAL_UI_DICTIONARY = {
    "ml": {
        "result_labels": {
            "plant": "ചെടി",
            "disease": "രോഗം",
            "confidence": "ഉറപ്പ്",
            "severity": "തീവ്രത",
            "source": "ഉറവിടം",
        },
        "treatment_title": "ചികിത്സാ പദ്ധതി",
        "result_values": {
            "Healthy": "ആരോഗ്യമുള്ളത്",
            "Moderate": "മിതമായ",
            "High": "കൂടുതൽ",
            "None": "ഇല്ല",
        },
        "kind_labels": {
            "Plant": "ചെടി:",
            "Disease": "രോഗം:",
            "Severity": "തീവ്രത:",
            "Symptoms": "ലക്ഷണങ്ങൾ:",
            "Possible Causes": "സാധ്യമായ കാരണങ്ങൾ:",
        },
    },
}
```

8.3 APPENDIX C. SOURCE CODE (Page 27 of 78)

```
        "Treatment::": "ചികിത്സാ രീതികൾ:",
        "Prevention::": "പ്രതിരോധ മാർഗങ്ങൾ:",
    }
}

def _get_model_path():
    configured_path = getattr(settings, "PLANT_DISEASE_MODEL_PATH", "")
    if configured_path:
        return Path(configured_path)

    default_h5 = Path(settings.BASE_DIR) / "model.h5"
    default_keras = (
        Path(settings.BASE_DIR)
        / "ml_testing"
        / "trained_models"
        / "plant_disease_mobilenetv2.keras"
    )

    if default_h5.exists():
        return default_h5
    return default_keras
```

8.3 APPENDIX C. SOURCE CODE (Page 28 of 78)

```
def _get_class_names_path():
    configured_path = getattr(settings,
"PLANT_DISEASE_CLASS_NAMES_PATH", "")
    if configured_path:
        return Path(configured_path)

    return (
        Path(settings.BASE_DIR)
        / "ml_testing"
        / "trained_models"
        / "plant_disease_mobilenetv2_class_names.json"
    )

def _get_ml_runtime_python():
    configured_path = getattr(settings, "ML_RUNTIME_PYTHON", "")
    if configured_path:
        return Path(configured_path)

    scripts_dir = "Scripts" if os.name == "nt" else "bin"
    executable = "python.exe" if os.name == "nt" else "python"
    return Path(settings.BASE_DIR) / "ml_venv" / scripts_dir /
executable
```

8.3 APPENDIX C. SOURCE CODE (Page 29 of 78)

```
def _get_ml_runtime_predict_script():
    configured_path = getattr(settings, "ML_RUNTIME_PREDICT_SCRIPT", "")
    if configured_path:
        return Path(configured_path)

    return Path(settings.BASE_DIR) / "ml_testing" / "predict_image.py"

def _get_image_size():
    configured_size = getattr(settings, "PLANT_DISEASE_IMAGE_SIZE",
(224, 224))
    return tuple(configured_size)

def _format_disease_name(label):
    cleaned = label.replace("___", " ").replace("__", " ").replace("_",
" ")
    cleaned = re.sub(r"\s+", " ", cleaned)
    return cleaned.strip()

def _clean_prediction_text(value, fallback="Unknown"):
    normalized = str(value or "").strip()
```

8.3 APPENDIX C. SOURCE CODE (Page 30 of 78)

```
normalized = normalized.strip(" .:;,'\\"`")
normalized = re.sub(r"^[\-\\*\s]+", "", normalized)
normalized = re.sub(r"\s+", " ", normalized)
return normalized or fallback

def _extract_json_fragment(text):
    start = text.find("{")
    end = text.rfind("}")
    if start == -1 or end == -1 or end <= start:
        return ""
    return text[start : end + 1]

def _normalize_gemini_prediction(raw_response):
    if not raw_response:
        raise RuntimeError("Gemini returned an empty response.")

    payload = None
    json_fragment = _extract_json_fragment(raw_response)
    if json_fragment:
        try:
            payload = json.loads(json_fragment)
```

8.3 APPENDIX C. SOURCE CODE (Page 31 of 78)

```
except json.JSONDecodeError:
    payload = None

plant_name = "Unknown"
disease_name = "Uncertain"
status = ""

if isinstance(payload, dict):
    plant_name = payload.get("plant_name") or payload.get("plant")
    or payload.get("crop") or plant_name
    disease_name = payload.get("disease_name") or
    payload.get("disease") or payload.get("result") or disease_name
    status = str(payload.get("status") or "").strip().lower()
else:
    plant_match = re.search(r"plant(?:_name)?\s*:\s*(.+)",
raw_response, flags=re.IGNORECASE)
    disease_match = re.search(r"disease(?:_name)?\s*:\s*(.+)",
raw_response, flags=re.IGNORECASE)
    if plant_match:
        plant_name = plant_match.group(1)
    if disease_match:
        disease_name = disease_match.group(1)
    elif not plant_match:
        disease_name = raw_response

    plant_name = _clean_prediction_text(plant_name, fallback="Unknown")
    disease_name = _clean_prediction_text(disease_name,
fallback="Uncertain")
```

8.3 APPENDIX C. SOURCE CODE (Page 32 of 78)

```
        if disease_name.lower() in {"healthy leaf", "no visible disease",
    "no disease"}:
            disease_name = "Healthy"
        if status == "healthy":
            disease_name = "Healthy"

        if plant_name.lower() in {"uncertain", "unknown plant", "not sure"}:
            plant_name = "Unknown"

        if status == "uncertain" or disease_name.lower() in {"uncertain",
    "unknown", "not sure"}:
            raise RuntimeError("Gemini could not confidently identify the
    disease from this image.")

    return {
        "plant_name": plant_name,
        "disease": disease_name,
    }

def _parse_local_model_label(raw_label):
    label = str(raw_label or "").strip()
    if "___" in label:
        plant_raw, disease_raw = label.split("___", 1)
    elif "_" in label:
```

8.3 APPENDIX C. SOURCE CODE (Page 33 of 78)

```
    plant_raw, disease_raw = label.split("_", 1)
    else:
        return {"plant_name": "Unknown", "disease":
_format_disease_name(label) or "Unknown"}

    plant_name = _format_disease_name(plant_raw) or "Unknown"
    disease_name = _format_disease_name(disease_raw) or "Unknown"

    plant_prefix = f"{plant_name} "
    if disease_name.lower().startswith(plant_prefix.lower()):
        disease_name = disease_name[len(plant_prefix):].strip()

    if disease_name.lower() == "healthy":
        disease_name = "Healthy"

    return {
        "plant_name": plant_name,
        "disease": disease_name or "Unknown",
    }

def _build_local_prediction(raw_label, confidence):
    parsed_prediction = _parse_local_model_label(raw_label)
    return {
```

8.3 APPENDIX C. SOURCE CODE (Page 34 of 78)

```
        "plant_name": parsed_prediction["plant_name"],
        "disease": parsed_prediction["disease"],
        "confidence": round(confidence * 100, 2) if confidence is not
None else None,
        "source": "local_model",
    }

def _lookup_disease_record(disease_name, plant_name=None):
    if not disease_name:
        return None

    queryset = Disease.objects.all()
    if plant_name and plant_name.lower() != "unknown":
        queryset = queryset.filter(crop__name__iexact=plant_name)

    return
    queryset.select_related("crop").filter(name__iexact=disease_name).first()

def _is_healthy_prediction(disease_name):
    return "healthy" in (disease_name or "").lower()

def _split_guidance_points(text):
```

8.3 APPENDIX C. SOURCE CODE (Page 35 of 78)

```
if not text:
    return []

segments = re.split(r"[\r\n;]+", text)
points = [segment.strip(" -•\t") for segment in segments if
segment.strip()]
return points or [text.strip()]

... [code truncated for report length] ...

# === FILE: dashboard/models.py ===
from django.db import models
from django.conf import settings

class FarmLocation(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="farm_location"
    )
    latitude = models.FloatField()
    longitude = models.FloatField()
    city_name = models.CharField(max_length=100, blank=True)
```

8.3 APPENDIX C. SOURCE CODE (Page 36 of 78)

```
def __str__(self):
    return f"{self.user.username}'s Farm - {self.city_name or
'Unknown Location'}"

class AgriculturalDataset(models.Model):
    name = models.CharField(max_length=200)
    description = models.TextField()
    dataset_file = models.FileField(upload_to="datasets/")
    uploaded_by = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        blank=True
    )
    uploaded_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-uploaded_at"]

    def __str__(self):
        return self.name
```

8.3 APPENDIX C. SOURCE CODE (Page 37 of 78)

```
# === FILE: dashboard/urls.py ===
from django.urls import path
from . import views

app_name = 'dashboard'

urlpatterns = [
    path('', views.dashboard_home, name='home'),
    path('section/<str:section_key>/', views.dashboard_section,
name='section'),
    path('update-location/', views.update_location,
name='update_location'),
    path('upload-dataset/', views.upload_dataset,
name='upload_dataset'),
]

# === FILE: dashboard/views.py ===
import math
from datetime import timedelta

from django.contrib.auth.decorators import login_required
from django.db.models import Avg, Count
from django.shortcuts import render, redirect
from django.utils import timezone
```

8.3 APPENDIX C. SOURCE CODE (Page 38 of 78)

```
from account.utils import get_leaf_quota_summary
from detection.models import LeafDiagnosis
from .models import FarmLocation, AgriculturalDataset
from .utils import get_weather_forecast_and_alerts

DASHBOARD_SECTION_CONFIG = {
    "overview": {
        "key": "overview",
        "label": "Overview",
        "template": "dashboard/partials/_overview.html",
    },
    "analytics": {
        "key": "analytics",
        "label": "Analytics",
        "template": "dashboard/partials/_analytics.html",
    },
    "risk": {
        "key": "risk",
        "label": "Risk Board",
        "template": "dashboard/partials/_risk.html",
    },
    "insights": {
```

8.3 APPENDIX C. SOURCE CODE (Page 39 of 78)

```
        "key": "insights",
        "label": "Insights",
        "template": "dashboard/partials/_insights.html",
    },
    "workflow": {
        "key": "workflow",
        "label": "Workflow",
        "template": "dashboard/partials/_workflow.html",
    },
    "history": {
        "key": "history",
        "label": "Scan History",
        "template": "dashboard/partials/_history.html",
    },
    "weather": {
        "key": "weather",
        "label": "Weather Alerts",
        "template": "dashboard/partials/_weather.html",
    },
    "datasets": {
        "key": "datasets",
        "label": "Historical Datasets",
        "template": "dashboard/partials/_datasets.html",
    },
}
```

8.3 APPENDIX C. SOURCE CODE (Page 40 of 78)

```
    },  
}  
  
def _round(value, digits=1):  
    return round(value, digits) if value is not None else None  
  
def _labeled_logs(queryset):  
    return  
    queryset.exclude(predicted_disease__isnull=True).exclude(predicted_disease  
  
def _issue_logs(queryset):  
    return  
    _labeled_logs(queryset).exclude(predicted_disease__icontains="healthy")  
  
def _healthy_logs(queryset):  
    return  
    _labeled_logs(queryset).filter(predicted_disease__icontains="healthy")  
  
def _safe_pct(numerator, denominator):  
    if not denominator:
```

8.3 APPENDIX C. SOURCE CODE (Page 41 of 78)

```
        return 0
    return round((numerator / denominator) * 100, 1)

def _health_score(queryset):
    avg_conf =
    queryset.exclude(confidence__isnull=True).aggregate(v=Avg("confidence"))
    ["v"]
    if avg_conf is not None:
        return round(avg_conf, 1)

    total = queryset.count()
    if not total:
        return None

    issue_count = _issue_logs(queryset).count()
    return round(((total - issue_count) / total) * 100, 1)

def _recovery_rate(queryset):
    total = queryset.count()
    if not total:
        return 0
    return _safe_pct(_healthy_logs(queryset).count(), total)
```

8.3 APPENDIX C. SOURCE CODE (Page 42 of 78)

```
def _build_demo_analytics(today):
    trend_30 = []
    heatmap_counts = {}

    for i in range(29, -1, -1):
        current_day = today - timedelta(days=i)
        position = 29 - i
        health = round(69 + (position * 0.58) + math.sin(position / 3.2)
* 4.6, 1)
        disease = round(31 - (position * 0.42) + math.cos(position /
4.1) * 3.2, 1)
        scans = max(4, 6 + ((position * 3) % 7))
        trend_30.append(
            {
                "date": current_day.strftime("%b %d"),
                "iso": current_day.isoformat(),
                "health": min(97, max(52, health)),
                "disease": min(48, max(6, disease)),
                "scans": scans,
            }
        )
```

8.3 APPENDIX C. SOURCE CODE (Page 43 of 78)

```
for i in range(83, -1, -1):
    current_day = today - timedelta(days=i)
    wave = 3.8 + math.sin(i / 5.5) * 2.6 + math.cos(i / 9.0) * 1.4
    heatmap_counts[current_day.isoformat()] = max(0, int(round(wave
+ (i % 4))))

return {
    "using_demo": True,
    "trend_30": trend_30,
    "trend_7": trend_30[-7:],
    "distribution": [
        {"label": "Healthy", "value": 44},
        {"label": "Leaf Spot", "value": 23},
        {"label": "Blight", "value": 18},
        {"label": "Rust", "value": 9},
        {"label": "Mildew", "value": 6},
    ],
    "plant_issues": [
        {"label": "Tomato Field A", "issues": 14},
        {"label": "Pepper Greenhouse", "issues": 11},
        {"label": "Potato Block C", "issues": 9},
        {"label": "Corn Plot 2", "issues": 6},
        {"label": "Apple Orchard East", "issues": 4},
```

8.3 APPENDIX C. SOURCE CODE (Page 44 of 78)

```
        ],
        "heatmap_counts": heatmap_counts,
    }

def _build_chart_data(diagnosis_logs):
    today = timezone.localdate()

... [code truncated for report length] ...

# === FILE: adminpanel/urls.py ===
from django.urls import path

from . import views

app_name = "adminpanel"

urlpatterns = [
    path("", views.admin_login_view, name="login"),
    path("logout/", views.admin_logout_view, name="logout"),
    path("dashboard/", views.dashboard_view, name="dashboard"),
    path("users/", views.users_view, name="users"),
```

8.3 APPENDIX C. SOURCE CODE (Page 45 of 78)

```

    path("messages/", views.messages_view, name="messages"),
    path("diagnoses/", views.diagnoses_view, name="diagnoses"),
        path("users/<int:user_id>/toggle-active/",
views.toggle_user_active_view, name="toggle_user_active"),
        path("users/<int:user_id>/toggle-staff/",
views.toggle_user_staff_view, name="toggle_user_staff"),
        path("messages/<int:message_id>/toggle-resolved/",
views.toggle_message_resolved_view, name="toggle_message_resolved"),

    # Resource Management
    path("crops/", views.crops_view, name="crops"),
    path("crops/add/", views.crop_upsert_view, name="crop_add"),
        path("crops/<int:crop_id>/edit/",    views.crop_upsert_view,
name="crop_edit"),
        path("crops/<int:crop_id>/delete/",    views.crop_delete_view,
name="crop_delete"),

    path("diseases/", views.diseases_view, name="diseases"),
        path("diseases/add/",    views.disease_upsert_view,
name="disease_add"),
        path("diseases/<int:disease_id>/edit/",    views.disease_upsert_view,
name="disease_edit"),
        path("diseases/<int:disease_id>/delete/",    views.disease_delete_view,
name="disease_delete"),

    path("datasets/", views.datasets_view, name="datasets"),
    path("datasets/<int:dataset_id>/delete/",    views.dataset_delete_view,
name="dataset_delete"),
]

```

8.3 APPENDIX C. SOURCE CODE (Page 46 of 78)

```
# === FILE: adminpanel/views.py ===
from datetime import timedelta

from django.contrib import messages
from django.contrib.auth import login, logout
from django.contrib.auth.models import User
from django.contrib.auth.decorators import user_passes_test
from django.core.paginator import Paginator
from django.db.models import Q
from django.shortcuts import get_object_or_404, redirect, render
from django.utils import timezone
from django.utils.http import url_has_allowed_host_and_scheme
from django.views.decorators.http import require_POST

from account.models import ContactMessage
from detection.models import LeafDiagnosis, Crop, Disease
from dashboard.models import AgriculturalDataset

from .forms import AdminLoginForm, CropForm, DiseaseForm

def staff_required(view_func):
```

8.3 APPENDIX C. SOURCE CODE (Page 47 of 78)

```
actual_decorator = user_passes_test(
    lambda user: user.is_authenticated and user.is_staff,
    login_url="adminpanel:login",
)
return actual_decorator(view_func)

def _safe_redirect(request, fallback):
    redirect_to = request.POST.get("next") or request.GET.get("next")
    if redirect_to and url_has_allowed_host_and_scheme(
        url=redirect_to,
        allowed_hosts={request.get_host()},
        require_https=request.is_secure(),
    ):
        return redirect_to
    return fallback

def admin_login_view(request):
    if request.user.is_authenticated and request.user.is_staff:
        return redirect("adminpanel:dashboard")
```

8.3 APPENDIX C. SOURCE CODE (Page 48 of 78)

```
next_url = request.POST.get("next") or request.GET.get("next", "")

if request.method == "POST":
    form = AdminLoginForm(request, data=request.POST)
    if form.is_valid():
        login(request, form.get_user())
        messages.success(request, "Admin access granted.")
        return redirect(_safe_redirect(request,
"adminpanel:dashboard"))
    else:
        form = AdminLoginForm(request)

return render(
    request,
    "adminpanel/login.html",
    {"form": form, "next_url": next_url},
)

@require_POST
def admin_logout_view(request):
    if request.user.is_authenticated:
        logout(request)
```

8.3 APPENDIX C. SOURCE CODE (Page 49 of 78)

```
        messages.success(request, "Admin session closed.")
    return redirect("adminpanel:login")

@staff_required
def dashboard_view(request):
    week_ago = timezone.now() - timedelta(days=7)
    users = User.objects.order_by("-date_joined")
    messages_qs = ContactMessage.objects.order_by("-created_at")
    diagnoses_qs = LeafDiagnosis.objects.order_by("-created_at")

    # Chart Data: Last 7 days diagnosis counts
    today = timezone.now().date()
    chart_labels = []
    chart_data = []
    for i in range(6, -1, -1):
        day = today - timedelta(days=i)
        count = LeafDiagnosis.objects.filter(created_at__date=day).count()
        chart_labels.append(day.strftime("%b %d"))
        chart_data.append(count)

    context = {
```

8.3 APPENDIX C. SOURCE CODE (Page 50 of 78)

```
        "total_users": users.count(),
        "active_users": users.filter(is_active=True).count(),
        "blocked_users": users.filter(is_active=False).count(),
        "staff_users": users.filter(is_staff=True).count(),
        "new_users_this_week":
users.filter(date_joined__gte=week_ago).count(),
        "unresolved_messages":
messages_qs.filter(is_resolved=False).count(),
        "total_diagnoses": diagnoses_qs.count(),
        "total_crops": Crop.objects.count(),
        "total_diseases": Disease.objects.count(),
        "total_datasets": AgriculturalDataset.objects.count(),
        "recent_users": users[:6],
        "recent_messages": messages_qs[:6],
        "recent_diagnoses": diagnoses_qs[:6],
        "chart_labels": chart_labels,
        "chart_data": chart_data,
    }
    return render(request, "adminpanel/dashboard.html", context)

@staff_required
def users_view(request):
    query = request.GET.get("q", "").strip()
```

8.3 APPENDIX C. SOURCE CODE (Page 51 of 78)

```
status = request.GET.get("status", "all")

users = User.objects.order_by("-date_joined")
if query:
    users = users.filter(
        Q(username__icontains=query)
        | Q(first_name__icontains=query)
        | Q(last_name__icontains=query)
        | Q(email__icontains=query)
    )

if status == "active":
    users = users.filter(is_active=True)
elif status == "blocked":
    users = users.filter(is_active=False)
elif status == "staff":
    users = users.filter(is_staff=True)

paginator = Paginator(users, 10)
page_obj = paginator.get_page(request.GET.get("page"))

return render(
```

8.3 APPENDIX C. SOURCE CODE (Page 52 of 78)

```
request,
"adminpanel/users.html",
{
    "page_obj": page_obj,
    "query": query,
    "status": status,
    "total_results": users.count(),
},
)

@staff_required
def messages_view(request):
    query = request.GET.get("q", "").strip()
    status = request.GET.get("status", "all")

    messages_qs = ContactMessage.objects.order_by("-created_at")
    if query:
        messages_qs = messages_qs.filter(
... [code truncated for report length] ...

# === FILE: templates/account/signup.html ===
```

8.3 APPENDIX C. SOURCE CODE (Page 53 of 78)

```
{% extends "base.html" %}
{% load static %}

{% block title %}Sign Up | PlantGuard{% endblock %}

{% block body_class %}auth-body{% endblock %}

{% block main_class %}w-full flex items-center justify-center py-16 px-4
sm:px-6 lg:px-8 min-h-[85vh]{% endblock %}

{% block content %}
<div class="w-full max-w-xl animate-fade-in-up">
  <div class="auth-glass-card">
    <!-- Brand logo -->
    <div class="flex items-center justify-center gap-3 mb-8">
      <div class="w-11 h-11 rounded-xl flex items-center justify-
center"
                                style="background:linear-
gradient(135deg,#00c062,#009650);    box-shadow:0    4px    20px
    rgba(0,192,98,0.3)">
        <i data-lucide="leaf" class="w-6 h-6 text-white"></i>
      </div>
      <span class="text-white font-extrabold text-2xl tracking-
tight">PlantGuard</span>
    </div>

    <!-- Header -->
```

8.3 APPENDIX C. SOURCE CODE (Page 54 of 78)

```
<div class="text-center mb-8">
  <h1 class="text-3xl font-bold auth-title mb-2">Create Your
Account</h1>
  <p class="text-sm auth-text leading-relaxed">
    Join PlantGuard and protect your crops. Get instant AI
crop health diagnoses using our secure passwordless login.
  </p>
</div>

{% if form.non_field_errors %}
  <div class="mb-5 rounded-xl border border-red-500/20 bg-
red-500/10 p-4 text-sm text-red-200">
    {% for error in form.non_field_errors %}
      <div class="flex items-center gap-2">
        <i data-lucide="alert-circle" class="w-4 h-4
flex-shrink-0"></i>
        <span>{{ error }}</span>
      </div>
    {% endfor %}
  </div>
{% endif %}

<!-- Form -->
<form method="post" class="space-y-5">
  {% csrf_token %}
```

8.3 APPENDIX C. SOURCE CODE (Page 55 of 78)

```

        <div class="grid grid-cols-2 gap-4">
            <div class="space-y-2">
                <label for="{{ form.first_name.id_for_label }}"
class="text-sm auth-label flex items-center gap-2">
                    <i data-lucide="user" class="w-4 h-4 text-
brand-400"></i> First Name
                </label>
                {{ form.first_name }}
                {% for error in form.first_name.errors %}
                    <p class="text-xs text-red-400 font-semibold
flex items-center gap-1 mt-1">
                        <i data-lucide="alert-triangle" class="w-3
h-3"></i> {{ error }}
                    </p>
                {% endfor %}
            </div>
            <div class="space-y-2">
                <label for="{{ form.last_name.id_for_label }}"
class="text-sm auth-label flex items-center gap-2">
                    <i data-lucide="user" class="w-4 h-4 text-
brand-400"></i> Last Name
                </label>
                {{ form.last_name }}
                {% for error in form.last_name.errors %}
                    <p class="text-xs text-red-400 font-semibold
flex items-center gap-1 mt-1">
                        <i data-lucide="alert-triangle" class="w-3
h-3"></i> {{ error }}
                    </p>
                {% endfor %}
            </div>
        </div>

```

8.3 APPENDIX C. SOURCE CODE (Page 56 of 78)

```

        </div>
    </div>

    <div class="space-y-2">
        <label for="{{ form.email.id_for_label }}" class="text-sm auth-label flex items-center gap-2">
            <i data-lucide="mail" class="w-4 h-4 text-brand-400"></i> Email Address
        </label>
        {{ form.email }}
        {% for error in form.email.errors %}
            <p class="text-xs text-red-400 font-semibold flex items-center gap-1 mt-1">
                <i data-lucide="alert-triangle" class="w-3 h-3"></i> {{ error }}
            </p>
        {% endfor %}
        <p class="text-xs auth-muted mt-1">We will send a 6-digit verification code to this email.</p>
    </div>

    <button type="submit" class="btn btn-primary btn-lg w-full justify-center mt-2"
        style="height:52px; font-size:0.95rem">
        <i data-lucide="send" class="w-4 h-4"></i> Send OTP Code
    </button>
</form>

```

8.3 APPENDIX C. SOURCE CODE (Page 57 of 78)

```
<!-- Help Info list / Platform Benefits -->
<div class="mt-8 space-y-3.5 pt-6 border-t border-white/5">
  <div class="flex items-start gap-3 text-xs auth-muted">
    <i data-lucide="scan-line" class="w-4 h-4 text-brand-400
mt-0.5 flex-shrink-0"></i>
      <span><strong>20 free leaf scans</strong> included to
instantly diagnose plant diseases.</span>
    </div>
    <div class="flex items-start gap-3 text-xs auth-muted">
      <i data-lucide="brain-circuit" class="w-4 h-4 text-
brand-400 mt-0.5 flex-shrink-0"></i>
      <span>AI Dual Engine: Leverages localized <strong>CNN +
Google Gemini</strong>.</span>
    </div>
    <div class="flex items-start gap-3 text-xs auth-muted">
      <i data-lucide="volume-2" class="w-4 h-4 text-brand-400
mt-0.5 flex-shrink-0"></i>
      ... [code truncated for report length] ...

# === FILE: templates/account/login.html ===
{% extends "base.html" %}
{% load static %}

{% block title %}Login | PlantGuard{% endblock %}

{% block body_class %}auth-body{% endblock %}
```

8.3 APPENDIX C. SOURCE CODE (Page 58 of 78)

```
{% block main_class %}w-full flex items-center justify-center py-16 px-4
sm:px-6 lg:px-8 min-h-[85vh]{% endblock %}

{% block content %}
<div class="w-full max-w-lg animate-fade-in-up">
  <div class="auth-glass-card">
    <!-- Brand logo -->
    <div class="flex items-center justify-center gap-3 mb-8">
      <div class="w-11 h-11 rounded-xl flex items-center justify-
center"
                                style="background:linear-
gradient(135deg,#00c062,#009650);    box-shadow:0    4px    20px
rgba(0,192,98,0.3)">
        <i data-lucide="leaf" class="w-6 h-6 text-white"></i>
      </div>
      <span class="text-white font-extrabold text-2xl tracking-
tight">PlantGuard</span>
    </div>

    <!-- Header -->
    <div class="text-center mb-8">
      <h1 class="text-3xl font-bold auth-title mb-2">Welcome
Back</h1>

      <p class="text-sm auth-text leading-relaxed">
        Enter your email address and we'll send a secure one-
time code to log you in instantly.
      </p>
    </div>
  </div>
</div>
```

8.3 APPENDIX C. SOURCE CODE (Page 59 of 78)

```

        {% if form.non_field_errors %}
            <div class="mb-5 rounded-xl border border-red-500/20 bg-
red-500/10 p-4 text-sm text-red-200">
                {% for error in form.non_field_errors %}
                    <div class="flex items-center gap-2">
                        <i data-lucide="alert-circle" class="w-4 h-4
flex-shrink-0"></i>
                            <span>{{ error }}</span>
                        </div>
                    {% endfor %}
                </div>
            {% endif %}

        <!-- Form -->
        <form method="post" class="space-y-6">
            {% csrf_token %}
            {% if next_url %}<input type="hidden" name="next" value="{{
next_url }}">{% endif %}

            <div class="space-y-2">
                <label for="{{ form.email.id_for_label }}" class="text-
sm auth-label flex items-center gap-2">
                    <i data-lucide="mail" class="w-4 h-4 text-
brand-400"></i> Email Address
                </label>
                {{ form.email }}
                {% for error in form.email.errors %}

```

8.3 APPENDIX C. SOURCE CODE (Page 60 of 78)

```

                <p class="text-xs text-red-400 font-semibold flex
items-center gap-1 mt-1">
                    <i data-lucide="alert-triangle" class="w-3
h-3"></i> {{ error }}
                </p>
            {% endfor %}
        </div>

        <button type="submit" class="btn btn-primary btn-lg w-full
justify-center"
            style="height:52px; font-size:0.95rem">
            <i data-lucide="send" class="w-4 h-4"></i> Send OTP Code
        </button>
    </form>

    <!-- Help Info list -->
    <div class="mt-8 space-y-3.5 pt-6 border-t border-white/5">
        <div class="flex items-start gap-3 text-xs auth-muted">
            <i data-lucide="shield-check" class="w-4 h-4 text-
brand-400 mt-0.5 flex-shrink-0"></i>
            <span>Passwordless login – no insecure passwords to
manage or remember.</span>
        </div>
        <div class="flex items-start gap-3 text-xs auth-muted">
            <i data-lucide="clock" class="w-4 h-4 text-brand-400
mt-0.5 flex-shrink-0"></i>
            <span>Secure codes expire automatically within 5 minutes
for safety.</span>
        </div>
    </div>

```

8.3 APPENDIX C. SOURCE CODE (Page 61 of 78)

```
</div>

<!-- Footer -->
<div class="mt-8 pt-6 border-t border-white/5 text-center">
  <p class="text-sm auth-muted">
    Don't have an account yet?
    <a href="{% url 'account:signup' %}" class="font-bold
text-brand-400 hover:text-brand-300 transition-colors ml-1">Sign up
free</a>
  </p>
</div>
</div>
{% endblock %}

# === FILE: templates/account/verify_otp.html ===
{% extends "base.html" %}
{% load static %}

{% block title %}Verify OTP | PlantGuard{% endblock %}

{% block body_class %}auth-body{% endblock %}
```

8.3 APPENDIX C. SOURCE CODE (Page 62 of 78)

```
{% block main_class %}w-full flex items-center justify-center py-16 px-4
sm:px-6 lg:px-8 min-h-[85vh]{% endblock %}

{% block content %}
<div class="w-full max-w-md animate-fade-in-up">
  <div class="auth-glass-card">
    <!-- Header -->
    <div class="text-center mb-8">
      <div class="w-16 h-16 rounded-2xl mx-auto mb-5 flex items-
center justify-center relative"
                                style="background:linear-
gradient(135deg,#00c062,#009650);    box-shadow:0    4px    20px
rgba(0,192,98,0.3)">
        <i data-lucide="shield-check" class="w-8 h-8 text-
white"></i>
        <!-- Pulse ring -->
        <div class="absolute inset-0 rounded-2xl animate-ping
opacity-20"
            style="background:#00c062"></div>
      </div>
      <h1 class="text-3xl font-bold auth-title mb-2">Verify OTP</
h1>
      <p class="text-sm auth-text leading-relaxed">
        We sent a 6-digit verification code to
      </p>
      <p class="text-sm font-bold text-brand-400 mt-1.5 flex
items-center justify-center gap-1.5">
        <i data-lucide="mail" class="w-4 h-4"></i>
        {{ email }}
      </p>
    </div>
  </div>
</div>
```

8.3 APPENDIX C. SOURCE CODE (Page 63 of 78)

```

    </div>

    <!-- Purpose badge -->
    <div class="flex justify-center mb-6">
        <span class="inline-flex items-center gap-1.5 px-3.5 py-1.5
rounded-full text-xs font-bold uppercase tracking-widest
        {% if purpose == 'signup' %}bg-emerald-500/15 text-
brand-300 border border-emerald-500/30
        {% else %}bg-brand-500/15 text-brand-300 border border-
brand-500/30{% endif %}">
            <i data-lucide="{% if purpose == 'signup' %}user-plus{%
else %}log-in{% endif %}" class="w-3.5 h-3.5"></i>
            {% if purpose == "signup" %}Creating account{% else %}
Logging in{% endif %}
        </span>
    </div>

    {% if form.non_field_errors %}
        <div class="mb-5 rounded-xl border border-red-500/20 bg-
red-500/10 p-4 text-sm text-red-200">
            {% for error in form.non_field_errors %}
                <div class="flex items-center gap-2">
                    <i data-lucide="alert-circle" class="w-4 h-4
flex-shrink-0"></i>
                    <span>{{ error }}</span>
                </div>
            {% endfor %}
        </div>
    {% endif %}

```

8.3 APPENDIX C. SOURCE CODE (Page 64 of 78)

```

<!-- Form -->
<form method="post" class="space-y-6">
  {% csrf_token %}

  <div class="space-y-2">
    <label for="{{ form.otp.id_for_label }}" class="text-sm
auth-label text-center block">OTP Code</label>
    {{ form.otp }}
    {% for error in form.otp.errors %}
      <p class="text-xs text-red-400 font-semibold text-
center mt-1.5 flex items-center justify-center gap-1">
        <i data-lucide="alert-triangle" class="w-3
h-3"></i> {{ error }}
      </p>
    {% endfor %}
  </div>

  <button type="submit" class="btn btn-primary btn-block btn-
lg w-full justify-center"
    style="height:52px; font-size:0.95rem">
    <i data-lucide="check-circle" class="w-4 h-4"></i>
    {% if purpose == "signup" %}Create Account{% else %}
Login{% endif %}
  </button>
</form>

```

8.3 APPENDIX C. SOURCE CODE (Page 65 of 78)

```

        <!-- Timer & resend info -->
        <div class="mt-6 text-center space-y-3">
            <div class="flex items-center justify-center gap-2 text-xs
auth-muted">
                <i data-lucide="clock" class="w-3.5 h-3.5 text-
brand-400"></i>
                <span>Code expires in 5 minutes</span>
            </div>
            <p class="text-sm auth-muted">
                Didn't receive the code?
                <a href="{% if purpose == 'signup' %}{% url
'account:signup' %}{% else %}{% url 'account:login' %}{% endif %}"
                    class="font-bold text-brand-400 hover:text-brand-300
transition-colors ml-1">Resend OTP</a>
            </p>
        </div>

        <!-- Info box -->
        <div class="mt-8 rounded-xl p-4 auth-info-box">
            <div class="flex items-start gap-3">
                <i data-lucide="info" class="w-4 h-4 text-brand-400
mt-0.5 flex-shrink-0"></i>
                <div class="text-xs auth-text leading-relaxed">
                    <p class="font-bold text-white mb-1">Development
Tip:</p>
                    <p class="auth-muted">The OTP is printed directly in
your <strong>terminal/console</strong> where the Django server is
running. Check there for the 6-digit code.</p>
                </div>
            </div>
        </div>

```

8.3 APPENDIX C. SOURCE CODE (Page 66 of 78)

```
        </div>
    </div>
</div>
{% endblock %}

# === FILE: templates/detection/upload.html ===
{% extends "base.html" %}

{% block title %}Upload Scan | Plant Disease System{% endblock %}

{% block extra_head %}
<link          href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/
bootstrap.min.css" rel="stylesheet">
{% endblock %}

{% block content %}
<section class="py-2">
    <div class="container px-0">
        <div class="row justify-content-center">
            <div class="col-lg-8 col-xl-7">
                <div class="card border-0 shadow-lg rounded-4 overflow-
hidden">
                    <div class="card-body p-4 p-md-5">
```

8.3 APPENDIX C. SOURCE CODE (Page 67 of 78)

```

        <div class="mb-4">
            <span class="badge text-bg-success-subtle
border border-success-subtle text-success-emphasis rounded-pill px-3
py-2">
                Disease Scan Upload
            </span>
            <h1 class="display-6 fw-bold mt-3
mb-2">Upload a leaf image for analysis</h1>
            <p class="text-secondary mb-0">
                Add a clear image of the affected leaf.
The AI prediction step will be connected next,
                so this form currently stores the scan
and prepares it for diagnosis.
            </p>
        </div>

        <form method="post" enctype="multipart/form-
data" class="mt-4">
            {% csrf_token %}

            {% if form.non_field_errors %}
                <div class="alert alert-danger">
                    {% for error in
form.non_field_errors %}
                        <div>{{ error }}</div>
                    {% endfor %}
                </div>
            {% endif %}

```

8.3 APPENDIX C. SOURCE CODE (Page 68 of 78)

```

<div class="mb-4">
    <label for="{{
form.leaf_image.id_for_label }}" class="form-label fw-semibold">
        {{ form.leaf_image.label }}
    </label>
    {{ form.leaf_image }}
    {% for error in form.leaf_image.errors
%}
        <div class="text-danger small
mt-2">{{ error }}</div>
    {% endfor %}
    <div class="form-text">Use JPG, JPEG, or
PNG images for best results.</div>
    </div>

    <div class="d-flex flex-column flex-sm-row
gap-3">
        <button type="submit" class="btn btn-
success btn-lg px-4">
            Upload Scan
        </button>
        <a href="{% url 'dashboard:home' %}"
class="btn btn-outline-secondary btn-lg px-4">
            Back to Dashboard
        </a>
    </div>
</form>
</div>
</div>

```

8.3 APPENDIX C. SOURCE CODE (Page 69 of 78)

```
        </div>
    </div>
</div>
</section>
{% endblock %}

# === FILE: templates/dashboard/partials/_overview.html ===
<section class="space-y-6">
    <div class="flex flex-col gap-3 lg:flex-row lg:items-end lg:justify-
between">
        <div>
            <p class="text-xs font-semibold uppercase tracking-[0.22em]
text-emerald-700">Overview</p>
            <h2 class="mt-2 text-3xl font-black text-gray-900">Your crop
dashboard at a glance.</h2>
            <p class="mt-2 max-w-3xl text-sm text-gray-600">
                Start here for the core account numbers before moving
into analytics, alerts, or scan history.
            </p>
        </div>
        <div class="inline-flex items-center gap-2 rounded-full border
border-emerald-100 bg-emerald-50 px-4 py-2 text-xs font-bold uppercase
tracking-[0.22em] text-emerald-700">
            <span class="h-2 w-2 rounded-full bg-emerald-500"></span>
            Command center
        </div>
    </div>
</div>
```

8.3 APPENDIX C. SOURCE CODE (Page 70 of 78)

```

<section class="grid gap-4 sm:grid-cols-2 xl:grid-cols-4">
  <article class="card card-hover p-6">
    <p class="text-xs font-semibold uppercase tracking-[0.22em]
text-emerald-700">Total scans</p>
    <h2 class="mt-3 text-3xl font-black text-gray-900">{{
dashboard_metrics.diagnosis_count }}</h2>
    <p class="mt-2 text-sm text-gray-600">All completed leaf
diagnoses saved to this account.</p>
  </article>
  <article class="card card-hover p-6">
    <p class="text-xs font-semibold uppercase tracking-[0.22em]
text-emerald-700">Plants tracked</p>
    <h2 class="mt-3 text-3xl font-black text-gray-900">{{
dashboard_metrics.tracked_plants }}</h2>
    <p class="mt-2 text-sm text-gray-600">Distinct plant names
identified in your recent scan history.</p>
  </article>
  <article class="card card-hover p-6">
    <p class="text-xs font-semibold uppercase tracking-[0.22em]
text-emerald-700">Gemini verified</p>
    <h2 class="mt-3 text-3xl font-black text-gray-900">{{
dashboard_metrics.verified_count }}</h2>
    <p class="mt-2 text-sm text-gray-600">Scans validated
through Gemini review after model prediction.</p>
  </article>
  <article class="card card-hover p-6">
    <p class="text-xs font-semibold uppercase tracking-[0.22em]
text-emerald-700">Avg confidence</p>
    <h2 class="mt-3 text-3xl font-black text-gray-900">
    {% if dashboard_metrics.avg_confidence is not None %}
      {{ dashboard_metrics.avg_confidence }}%

```

8.3 APPENDIX C. SOURCE CODE (Page 71 of 78)

```

        {% else %}
            --
        {% endif %}
    </h2>
    <p class="mt-2 text-sm text-gray-600">Average confidence
across scans that already have a score.</p>
    </article>
</section>
</section>

# === FILE: templates/dashboard/partials/_analytics.html ===
<section data-analytics-dashboard class="space-y-6">
    <div class="flex flex-col gap-3 lg:flex-row lg:items-end lg:justify-
between">
        <div>
            <p class="text-xs font-semibold uppercase tracking-[0.22em]
text-emerald-700">Decision Intelligence</p>
            <h2 class="mt-2 text-3xl font-black text-gray-900">Analytics
built for action, not just display.</h2>
            <p class="mt-2 max-w-3xl text-sm text-gray-600">
                Visualize trend direction, disease mix, risk
concentration, and scan behavior so the dashboard becomes a real crop-
health control room.
            </p>
        </div>
        <div class="inline-flex items-center gap-2 rounded-full border
border-emerald-100 bg-emerald-50 px-4 py-2 text-xs font-bold uppercase
tracking-[0.22em] text-emerald-700">
            <span class="h-2 w-2 rounded-full bg-emerald-500"></span>

```

8.3 APPENDIX C. SOURCE CODE (Page 72 of 78)

```

        {% if analytics.using_demo %}Preview analytics{% else %}Live
account analytics{% endif %}
    </div>
</div>

<section class="grid gap-4 md:grid-cols-2 xl:grid-cols-4">
    {% for card in weekly_cards %}
        <article class="group relative overflow-hidden rounded-
[1.75rem] border border-gray-200 bg-white p-5 shadow-
[0_18px_44px_rgba(17,24,17,0.06)] transition-all duration-300 hover:-
translate-y-1 hover:shadow-[0_24px_58px_rgba(17,24,17,0.11)]">
            <div class="absolute inset-x-0 top-0 h-1
                {% if card.tone == 'red' %}
                    bg-gradient-to-r from-rose-500 via-red-400 to-
amber-300
                {% elif card.tone == 'amber' %}
                    bg-gradient-to-r from-amber-400 via-lime-400 to-
emerald-400
                {% elif card.tone == 'slate' %}
                    bg-gradient-to-r from-slate-300 via-slate-200
to-emerald-300
                {% else %}
                    bg-gradient-to-r from-emerald-500 via-green-400
to-lime-300
                {% endif %}">
            </div>
            <div class="flex items-start justify-between gap-4">
                <div>
                    <p class="text-xs font-semibold uppercase
tracking-[0.22em] text-emerald-700">{{ card.title }}</p>
                    <h3 class="mt-3 text-3xl font-black text-
gray-900">{{ card.value }}</h3>

```

8.3 APPENDIX C. SOURCE CODE (Page 73 of 78)

```

        </div>
        <span class="inline-flex items-center gap-1 rounded-
full px-3 py-1 text-xs font-bold
        {% if card.tone == 'red' %}
            bg-red-50 text-red-600
        {% elif card.tone == 'amber' %}
            bg-amber-50 text-amber-700
        {% elif card.tone == 'slate' %}
            bg-slate-100 text-slate-600
        {% else %}
            bg-emerald-50 text-emerald-700
        {% endif %}">
            <i data-lucide="{% if card.trend == 'up' %}
trending-up{% else %}trending-down{% endif %}" class="h-3.5 w-3.5"></i>
            {{ card.delta }}
        </span>
    </div>
    <div class="mt-5 dashboard-chart-shell is-compact">
        <div class="dashboard-chart-placeholder">Loading
sparkline...</div>
        <canvas id="sparkline-{{ card.id }}" data-sparkline-
key="{{ card.id }}"></canvas>
    </div>
</article>
{% endfor %}
</section>

```

8.3 APPENDIX C. SOURCE CODE (Page 74 of 78)

```

<section class="grid gap-6 xl:grid-cols-[1.65fr_1fr]">
  <article class="card p-6 md:p-8">
    <div class="mb-6 flex flex-col gap-4 lg:flex-row lg:items-
start lg:justify-between">
      <div>
        <p class="text-xs font-semibold uppercase tracking-
[0.22em] text-emerald-700">Crop Health Trends</p>
        <h2 class="mt-2 text-2xl font-black text-
gray-900">Scan results over time</h2>
        <p class="mt-2 max-w-2xl text-sm text-gray-600">
          Compare overall health score against disease-
detected scan rate and switch between the last 7 and 30 days.
        </p>
      </div>
      <div class="inline-flex items-center gap-2 rounded-full
border border-gray-200 bg-gray-50 p-1">
        <button type="button" id="trendToggle7"
class="dashboard-toggle rounded-full border border-transparent px-4 py-2
text-xs font-bold text-gray-600 transition-all" data-range="7">Last 7
days</button>
        <button type="button" id="trendToggle30"
class="dashboard-toggle rounded-full border border-transparent px-4 py-2
text-xs font-bold text-gray-600 transition-all" data-range="30" data-
active="true">Last 30 days</button>
      </div>
    </div>
    <div class="dashboard-chart-shell h-[360px]">
      <div class="dashboard-chart-placeholder">Preparing crop
health chart...</div>
      <canvas id="cropHealthTrendChart"></canvas>
    </div>
  </article>

```

8.3 APPENDIX C. SOURCE CODE (Page 75 of 78)

```

<article class="card p-6 md:p-8">
  <div class="mb-6">
    <p class="text-xs font-semibold uppercase tracking-
[0.22em] text-emerald-700">Disease Distribution</p>
... [code truncated for report length] ...

# === FILE: templates/dashboard/partials/_history.html ===
<article class="card p-6 md:p-8">
  <div class="mb-6 flex items-start justify-between gap-4">
    <div>
      <p class="text-xs font-semibold uppercase tracking-[0.22em]
text-emerald-700">Recent scans</p>
      <h2 class="mt-2 text-2xl font-black text-gray-900">Latest
diagnosis activity</h2>
      <p class="mt-2 text-sm text-gray-600">Open the newest scans
quickly and keep an eye on model confidence and verification status.</p>
    </div>
    <a href="{% url 'detection:leaf_diagnosis' %}" class="btn btn-
primary">New scan</a>
  </div>

  <div class="space-y-4">
    {% for item in recent_diagnoses %}
      <article class="rounded-2xl border border-gray-200 bg-
gray-50 p-5">
        <div class="mb-3 flex flex-wrap items-center justify-
between gap-2">
          <div>
            <p class="text-sm font-bold text-gray-900">{{
item.predicted_disease|default:"Pending diagnosis" }}</p>

```

8.3 APPENDIX C. SOURCE CODE (Page 76 of 78)

```

                                <p class="text-xs text-gray-500">{{
item.created_at|date:"M d, Y h:i A" }}</p>
                                </div>
                                <span class="rounded-full px-3 py-1 text-xs font-
semibold {% if item.source == 'gemini_api' %}bg-emerald-100 text-
emerald-800{% else %}bg-white text-gray-700 border border-gray-200{%
endif %}">
                                {% if item.source == "gemini_api" %}Gemini
verified{% else %}Local CNN{% endif %}
                                </span>
                                </div>

                                <div class="grid gap-3 sm:grid-cols-3">
                                    <div>
                                        <p class="text-xs uppercase tracking-wide text-
gray-500">Leaf file</p>
                                        <p class="mt-1 text-sm font-semibold text-
gray-800">{{ item.original_filename|default:"Unnamed upload" }}</p>
                                    </div>
                                    <div>
                                        <p class="text-xs uppercase tracking-wide text-
gray-500">Plant</p>
                                        <p class="mt-1 text-sm font-semibold text-
gray-800">{{ item.plant_name|default:"Unknown" }}</p>
                                    </div>
                                    <div>
                                        <p class="text-xs uppercase tracking-wide text-
gray-500">Confidence</p>
                                        <p class="mt-1 text-sm font-semibold text-
gray-800">
                                            {% if item.confidence or item.confidence ==
0 %}
                                            {{ item.confidence }}%
                                            {% else %}

```

8.3 APPENDIX C. SOURCE CODE (Page 77 of 78)

```

                Not available
            {% endif %}
        </p>
    </div>
</div>
</article>
{% empty %}
    <div class="rounded-2xl border border-dashed border-gray-300
bg-gray-50 p-8 text-center">
        <p class="text-lg font-bold text-gray-900">No scans
yet</p>
        <p class="mt-2 text-sm text-gray-600">Start with your
first leaf upload and this area will turn into a quick review board.</p>
        <a href="{% url 'detection:leaf_diagnosis' %}"
class="btn btn-primary mt-4">Run first scan</a>
    </div>
{% endfor %}
</div>

{% if recent_diagnoses %}
    <div class="mt-6">
        <div class="mb-4 flex flex-wrap items-end justify-between
gap-3">
            <div>
                <p class="text-xs font-semibold uppercase tracking-
[0.22em] text-emerald-700">Compact Table</p>
                <h3 class="mt-2 text-xl font-black text-
gray-900">Recent scans in one quick grid</h3>
            </div>

```

8.3 APPENDIX C. SOURCE CODE (Page 78 of 78)

```

        <span class="rounded-full bg-emerald-50 px-3 py-1 text-
xs font-bold text-emerald-700">
            {{ recent_diagnoses|length }} latest record{{
recent_diagnoses|length|pluralize }}
        </span>
    </div>

    <div class="dashboard-table-wrap">
        <div class="overflow-x-auto">
            <table class="dashboard-table min-w-[720px]">
                <thead>
                    <tr>
                        <th>Result</th>
                        <th>Plant</th>
                        <th>Confidence</th>
                        <th>Source</th>
                        <th>Scanned</th>
                    </tr>
                </thead>
                <tbody>
                    {% for item in recent_diagnoses %}
                        <tr>
... [code truncated for report length] ...

```

9. BIBLIOGRAPHY

9.1 BIBLIOGRAPHY

The Plant Disease Detection system is an intelligent web application built with Python and Django to assist farmers and agriculturalists in keeping a digital record of their crop health scans. The programme gives users the ability to keep track of their daily leaf scans and breaks down their crop health by disease category. With the use of this programme, the user may keep track of their daily, weekly, and monthly crop disease trends. By providing users with a clear picture of their farm's health, the tool also aids in helping users stick to preventive treatment guidelines. The programme creates a report at the end of each month that displays the disease logs in a graphical format. To help users better understand their crop health patterns, the application also has a unique function that categorises the diseases. The programme also offers a user-accessible scan history that is available at any time. Users may simply control their crop issues and make wiser agricultural decisions with the aid of the plant disease detection programme.

Key academic and technical references utilized in the design and development of this project include:

1. Django Software Foundation (2026). Django Web Framework Documentation (v6.0). Retrieved from <https://docs.djangoproject.com/>
2. Google DeepMind & TensorFlow Team (2025). TensorFlow and Keras Machine Learning API References. Retrieved from <https://www.tensorflow.org/>
3. Howard, A. G., et al. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv preprint arXiv:1704.04861.
4. Open-Meteo Weather Forecasting API (2026). Meteorological Data and Alert Forecast Models. Retrieved from <https://open-meteo.com/>
5. PyMuPDF Development Group (2025). Document Rendering and Story Layout API Reference Guides. Retrieved from <https://pymupdf.readthedocs.io/>
6. SQLite Development Team (2026). SQL Database Management Runtime Architecture and Reference Manuals. Retrieved from <https://www.sqlite.org/>